

AXK Network: A Cryptographic Verification and Settlement Layer for Cross-Border Agricultural Commodity Trade in East Africa

BSE: Software Engineering

Oghenetejiri Jesse

Capstone

Supervisor: Junior Turatsinze

2026

DECLARATION

This project report is my original work, except where stated otherwise, and all external sources have been referenced or cited in this document. This work has not been presented for the award of a degree or for any similar purpose in any other university.

Signature ..... Date .....

Oghenetejiri Jesse

CERTIFICATION

The undersigned certifies that he has read and hereby recommends for acceptance by African Leadership University a report entitled "AXK Network: A Cryptographic Verification and Settlement Layer for Cross-Border Agricultural Commodity Trade in East Africa".

Signature ..... Date .....

Mr. Junior Turatsinze

Faculty,
Bachelor of Software Engineering,
African Leadership University

DEDICATION AND ACKNOWLEDGEMENT

This work is dedicated to the smallholder farmers and cooperative members of East Africa, whose harvests carry the region's food security and whose trade deserves to be trusted and paid on time, and to my family, whose support carried me through this degree.

I am grateful to my supervisor, Mr. Junior Turatsinze, for his guidance, his patience with each draft and his insistence on clear, honest work. I thank the faculty of the Bachelor of Software Engineering programme at African Leadership University for the foundation this project was built on. I thank the AXK Network team for the hosting, review and deployment support that put the system online, and the cooperative members, traders and inspectors who gave their time to test it and tell me what did and did not work. Any errors that remain are my own.

ABSTRACT

Intra-African trade remains low, and cross-border agricultural trade in East Africa is held back by two linked failures: a buyer cannot verify the quality, quantity and custody of a distant lot, and payment is slow, costly and hard to trust. Existing systems address one side only. Payment rails such as the Pan-African Payment and Settlement System settle money without verifying goods, and traceability platforms record goods without settling payment. This study designed, built and evaluated AXK Network, a verification and settlement layer that joins both in one workflow. Following a design science approach and an iterative development model, four Soroban smart contracts on the Stellar testnet (a lot registry, an escrow, a payment splitter and an attestation registry) were combined with a Frappe web application used by cooperatives, inspectors, carriers, port terminals and buyers. Only SHA-256 digests of trade records were written on-chain, and the escrow released payment only for lots that an authorised inspector had passed. The system was evaluated with 169 automated tests, 80 page loads across three browser engines and a simulated low-end phone, and fifteen trades on the Stellar testnet, six of which settled in 74 to 236 seconds from registration with every record matching the chain. Acceptance testing with 20 prospective users (cooperatives, buyers and carriers) gave mean scores out of 7 of 6.30 for perceived usefulness, 6.35 for perceived ease of use and 6.55 for trust, against a reported median of 30 days from agreeing a deal to receiving payment. The results show that commodity verification and settlement can be combined in a single, publicly checkable workflow, and point to a field pilot and non-custodial signing as the next steps.

TABLE OF CONTENTS

Chapter One: Introduction 12
1.1 Background to the Study 12
1.2 Statement of the Problem 12
1.3 Aim and Objectives of the Study 13
1.3.1 Aim 13
1.3.2 Specific Objectives 13
1.4 Research Questions 13
1.5 Project Scope 14
1.6 Significance and Justification 14
1.7 Research Budget 14
1.8 Research Timeline 14
1.9 Definition of Terms 15
Chapter Two: Literature Review 16
2.1 Introduction 16
2.2 Conceptual Review 16
2.3 Historical Background of the Research Topic 17
2.3.1 Origins and Early Context of the Research Problem 17
2.3.2 Foundational Theories 17
2.3.3 Key Milestones 18
2.3.4 Historical Gaps and Transition to the Current Study 18
2.4 Theoretical Review 18
2.4.1 Foundational Theories 18
2.5 Overview of Existing Systems 19
2.5.1 Strengths and Weaknesses of the Existing Systems 20
2.6 Review of Related Works (Empirical Review) 20
2.6.1 Summary of Reviewed Literature 22
2.7 General Comments 22
Chapter Three: System Analysis and Design 23
3.1 Introduction 23
3.2 Research Design 23
3.2.1 Data Sources 24
3.2.2 Baseline Model Description 25
3.2.3 Proposed Model Description 25

3.2.4 Metrics for Evaluation 26
3.3 System Analysis 27
3.3.1 Functional Requirements of the System 27
3.3.2 Non-functional Requirements of the System 28
3.3.3 Proposed Model Diagram 28
3.4 System Design 29
3.4.1 Architecture Diagram 29
3.4.2 Use Case Diagram 30
3.4.3 Database Design 31
3.4.4 Class Diagram 32
3.4.5 Sequence Diagram 33
3.5 Development Tools 35
Chapter Four: System Implementation, Results and Discussion of Findings 36
4.1 System Implementation 36
4.1.1 Introduction 36
4.1.2 Smart Contracts 36
4.1.3 Ledger Package and Application Layer 37
4.1.4 Deployment 38
4.2 Graphical View of the Project 38
4.2.1 Screenshots with Description 38
4.3 Results 47
4.4 Discussion of Findings 49
4.5 Testing 50
4.5.1 Introduction 50
4.5.2 Objective of Testing 51
4.5.3 Unit Testing Outputs 51
4.5.4 Validation Testing Outputs 56
4.5.5 Integration Testing Outputs 56
4.5.6 Functional and System Testing Results 57
4.5.7 Acceptance Testing Report 59
Chapter Five: Summary, Conclusion and Recommendations 62
5.1 Summary 62
5.2 Conclusion 62
5.3 Recommendations 63
5.4 Limitations 63

5.5 Contributions to the Body of Knowledge 64
5.6 Suggestions for Further Studies 64
References 65
Appendices 69
Appendix A: Project Links and How to Run the System 69
Appendix B: Evaluation Questionnaire Items 70
Appendix C: User Testing Report 71
Appendix D: Consent Form and Research Instruments 72
Appendix E: Smart Contract Security Guarantees 73

LIST OF TABLES

Table 1: Acronyms and abbreviations used in this report 11
Table 2: Research timeline 15
Table 3: Variables in the study 17
Table 4: Milestones relevant to the study 18
Table 5: Strengths and weaknesses of existing systems 20
Table 6: Summary of reviewed empirical literature 22
Table 7: Evaluation metrics and targets 26
Table 8: Functional requirements 27
Table 9: Non-functional requirements 28
Table 10: Development tools and versions 35
Table 11: Outcome of all lots run on the Stellar testnet 48
Table 12: Testing environments 51
Table 13: Requirements traceability and validation results 56
Table 14: Performance and resource use 58
Table 15: Compatibility results (31 October 2026) 58
Table 16: Participant profile 60
Table 17: Technology acceptance results 60
Table 18: Achievement of the specific objectives 62
Table 19: Questionnaire scale items 70
Table 20: Item-level questionnaire results (n = 20, scale 1 to 7) 71
Table 21: Causes of delay in participants' last trade (more than one could be chosen) 71
Table 22: Security guarantees enforced by the contracts and the tests that pin them 73

LIST OF FIGURES

Figure 1: Iterative and incremental development model used in the project 24
Figure 2: Life cycle of a lot in the proposed model 29
Figure 3: System architecture of AXK Network 30
Figure 4: Use case diagram 31
Figure 5: Entity relationship diagram (off-chain document and on-chain records) 32
Figure 6: Class diagram of the ledger package 33
Figure 7: Sequence diagram for funding an escrow 34
Figure 8: Sign-in page with one demo account per role 39
Figure 9: Overview of lots, work per role and custody progress 39
Figure 10: Cooperative screen for registering a lot 40
Figure 11: Inspector screen with measured values and pass or reject 41
Figure 12: Buyer screen for funding an escrow on an inspected lot 41
Figure 13: Logistics screen for dispatching a funded lot under a container seal 42
Figure 14: Port screen showing the expected seal and the gate-in form 43
Figure 15: Lot page with the nine-step custody timeline of a settled trade 43
Figure 16: Public verification page 44
Figure 17: Public page for a settled lot with digests, QR code and EPCIS export 45
Figure 18: Chain explorer reading live contract events from the Stellar testnet 46
Figure 19: Cooperative screen and public lot page on a phone 47
Figure 20: Time from registration and from escrow funding to settlement for each settled lot (seconds) 48
Figure 21: Seconds per step of one full trade on the Stellar testnet (lot 0015) 49
Figure 22: Automated tests per suite, all passing 49
Figure 23: Output of the 65 smart contract unit tests 52
Figure 24: Output of the 22 ledger package unit tests 53
Figure 25: Tamper alert when the arriving seal does not match the waybill 53
Figure 26: Funding refused because the buyer's balance is too low 54
Figure 27: Rejection refused until a reason is chosen 55
Figure 28: Output of the custody display, demo login and transaction field tests 55
Figure 29: Output of the end-to-end, EPCIS and ODK tests 57
Figure 30: Median screen ready time by browser engine and screen size 59

LIST OF ACRONYMS AND ABBREVIATIONS

Table 1: Acronyms and abbreviations used in this report

Acronym	Meaning
AfCFTA	African Continental Free Trade Area
API	Application Programming Interface
BI	Behavioural Intention (to use)
CLI	Command-Line Interface
EOU	Perceived Ease of Use
EPCIS	Electronic Product Code Information Services (GS1 standard)
ERD	Entity Relationship Diagram
FAO	Food and Agriculture Organization of the United Nations
GS1	Global Standards 1, the supply-chain standards organisation
HTTPS	Hypertext Transfer Protocol Secure
JSON	JavaScript Object Notation
MVP	Minimum Viable Product
ODK	Open Data Kit
OOP	Object-Oriented Programming
PAPSS	Pan-African Payment and Settlement System
PU	Perceived Usefulness
QR	Quick Response (code)
RPC	Remote Procedure Call
RWF	Rwandan Franc
SDLC	Software Development Life Cycle
SHA-256	Secure Hash Algorithm, 256-bit
SME	Small and Medium Enterprise
SUS	System Usability Scale
TAM	Technology Acceptance Model
TLS	Transport Layer Security
UI	User Interface
UML	Unified Modelling Language
VM	Virtual Machine
WASM	WebAssembly

CHAPTER ONE: INTRODUCTION

1.1 Background to the Study

Regional trade has long been treated as one of the main routes to growth and resilience for developing economies. The contrast between regions is wide. Trade between countries of the same region makes up roughly 60 percent of all trade in Asia and about 70 percent in Europe, while in Africa it has stayed between 14 and 18 percent, the lowest share of any region (African Export-Import Bank, 2024). The shortfall is not caused by a lack of goods. It reflects the absence of the institutions and digital systems that let traders exchange those goods across borders with confidence.

The African Continental Free Trade Area (AfCFTA) was created to close this gap. It joins 55 countries and about 1.3 billion people into a single market with a combined output of roughly USD 3.4 trillion (World Bank, 2020). World Bank modelling estimated that full implementation could raise regional income by 7 percent, close to USD 450 billion, and move about 30 million people out of extreme poverty by 2035. Early figures were encouraging: intra-African trade reached an estimated USD 220.3 billion in 2024, a rise of 12.4 percent on the previous year (African Export-Import Bank, 2024). Even so, non-tariff barriers still impose costs comparable to an 18 percent import tariff, and the annual trade finance gap is estimated at about USD 100 billion.

For trade in physical commodities, two frictions stand out. The first is verification: a buyer in one country has no reliable way to confirm, at the time it matters, the quantity, quality, origin and delivery status of goods coming from another. The second is settlement: payment is slow, expensive and often depends on foreign currency (Pan-African Payment and Settlement System, 2021). Both frictions weigh most heavily on agricultural trade, which most of the continent's population depends on. A large share of smallholder output is lost between the farm and the market, and much of that loss is institutional rather than biological. Producers who cannot prove the condition and origin of their goods to distant buyers, and who cannot be paid quickly and safely, sell early, sell locally and accept lower prices (Affognon et al., 2015; Food and Agriculture Organization of the United Nations, 2019).

This study was set in East Africa, in Rwanda, a landlocked economy whose growth depends heavily on cross-border trade along the Northern Corridor and with its neighbours, the Democratic Republic of Congo and Uganda. It addressed the absence of a shared, trusted system that verifies a commodity and settles its payment in the same workflow. The work formed part of AXK Network, a venture whose mission is to build the digital trust infrastructure that lets economies of the Global South trade with one another. The capstone produced an open-source verification and settlement layer that was built, tested on the Stellar test network and deployed online during the study.

1.2 Statement of the Problem

Despite the policy momentum behind the AfCFTA, cross-border agricultural trade in East Africa remained constrained by the lack of a system that verifies goods and settles payment within one trusted workflow. Two existing solutions came closest. The Pan-African Payment and Settlement System (PAPSS) settles intra-African payments in local currencies

in real time, but it settles payments rather than trades: it has no way to verify the underlying commodity, and it runs through central banks and licensed financial institutions, which leaves out the informal and smallholder actors who carry out most agricultural trade (Pan-African Payment and Settlement System, 2021). Correspondent banking and letters of credit, the second closest option, give payment assurance but are slow, costly and dependent on foreign-currency intermediaries, which puts them out of reach for cooperatives and small traders (World Bank, 2020).

The gap was therefore specific. No system available to smallholder cooperatives and small and medium traders combined verification of a commodity's quality, quantity and custody with fast, low-cost settlement in a single layer that both sides, and outsiders, could check. Without it, buyers either pay in advance and carry the risk of receiving poor or short goods, or cooperatives ship first and carry the risk of late or missing payment. Closing this gap speaks directly to the grand challenge of regional integration and to the opportunity in agriculture, and it was the problem this project set out to address.

1.3 Aim and Objectives of the Study

1.3.1 Aim

The aim of the study was to design and develop a cryptographic verification and settlement software layer that lets smallholder cooperatives and small and medium traders verify agricultural commodities and settle cross-border payments within a single trusted workflow, in order to reduce transaction friction, value loss and exclusion in East African agricultural trade.

1.3.2 Specific Objectives

1. To assess the verification and settlement failures that cause delay, loss and exclusion for cross-border agricultural traders connected to Kigali.
2. To design a cryptographic verification model that records a commodity's quantity, quality, provenance and delivery status as tamper-evident, shareable records.
3. To develop a functional minimum viable product that integrates the verification model with a local-currency settlement workflow and runs on widely available devices.
4. To test the minimum viable product with traders and cooperative members, measuring its effect on transaction completion time, trust and intention to adopt.
5. To evaluate the solution against defined success criteria and recommend a path for scaling across additional East African trade corridors.

1.4 Research Questions

1. What are the principal verification and settlement failures experienced by smallholder cross-border agricultural traders in the East African context?
2. How can a cryptographic layer represent commodity quality, quantity and provenance in a way that distant counterparties can trust?
3. To what extent does combining verification with local-currency settlement in a single workflow reduce transaction completion time and increase trader trust compared with current practice?

1.5 Project Scope

Geographically, the study focused on cross-border agricultural traders and cooperative actors operating out of Kigali, Rwanda, with trades oriented towards a single corridor, Rwanda to the Democratic Republic of Congo or Rwanda to Uganda. Functionally, the scope covered the full custody of one lot: registration by a cooperative, independent inspection, payment into escrow by a buyer, dispatch by a carrier under a container seal, gate-in at a port or inland terminal with a seal check, and settlement with a split of the proceeds between cooperative members. A public verification page and a standards-based export were also in scope.

Technically, the system was built and run on the Stellar test network with a test token standing in for the Rwandan franc, so no real money moved. Building a new payment rail, connecting to banks or mobile-money operators, obtaining regulatory licences, auditing the contracts for mainnet use and deploying across the continent were outside the scope of the study and are identified as future work in Chapter Five.

1.6 Significance and Justification

If the approach holds in practice, a cooperative can prove the condition and origin of its produce to a buyer across the border and receive payment quickly and safely, which reduces the pressure to sell early and locally at depressed prices. Because a significant part of postharvest loss in sub-Saharan Africa is institutional, even a modest reduction in value lost to failed verification and settlement translates into real gains in farmer income and food security (Affognon et al., 2015).

The main stakeholders are smallholder farmers and their cooperatives, small and medium cross-border traders, inspectors, carriers and terminal operators, and the buyers and agribusinesses that rely on verified produce. Secondary stakeholders include payment providers and the public bodies implementing the AfCFTA. By lowering the trust and settlement barriers that suppress intra-African trade, the project adds the commodity verification dimension that payment-focused infrastructure such as PAPSS lacks. Because the code was released under an open-source licence, other researchers and developers can inspect, reuse and extend it.

1.7 Research Budget

The study involved no research spending, and no budget was required. The software stack was open source, the Stellar test network charges no real fees, hosting was provided in kind by AXK Network on its existing server and domain, and participants were not paid.

1.8 Research Timeline

The work followed the timeline in Table 2. Dates for the build and testing come from the project's version history and test records.

Table 2: Research timeline

Period	Activity	Output
1 to 18 September 2026	Problem definition, literature review, research proposal	Research proposal submitted (18 September)
29 to 30 September	Smart contracts written and deployed to testnet; first end-to-end trades	Four contracts; first live trades on 30 September
1 to 15 October	Frappe application, role screens, public trace page, chain explorer	Working MVP; public open-source repository (15 October)
15 to 23 October	GS1 EPCIS export, ODK offline inspection, offline simulation, evaluation instruments and ethics application	Standards export; online simulation; survey, consent form and interview guide
28 to 31 October	Object-oriented restructuring, compatibility and performance testing, hosted deployment, screen redesign	Hosted system at capstone.axk.org; 169 automated tests; final product submission
8 November 2026	Acceptance testing with participants	Survey and interview data
November 2026	Final report and defence	This report

1.9 Definition of Terms

- **Lot:** a defined quantity of one commodity, such as 3,850 kg of parchment coffee, registered by a cooperative for sale.
- **Verification:** confirming that a lot's recorded quantity, quality, origin and custody are true and unchanged.
- **Settlement:** the transfer of payment from buyer to seller that completes a trade.
- **Escrow:** an arrangement in which payment is held by a neutral party, here a smart contract, until agreed conditions are met.
- **Smart contract:** a program stored on a blockchain that runs exactly as written and whose state anyone can inspect.
- **Digest (hash):** a fixed-length fingerprint of a record, computed with SHA-256; any change to the record produces a different digest.
- **Attestation:** a digest written to the ledger by an authorised party to commit to a record, such as a waybill, at a point in time.
- **Canonical JSON:** a single agreed way of writing a record as text (sorted keys, no spaces) so that the same record always produces the same digest.
- **Testnet:** a public test network that behaves like the live Stellar network but uses tokens with no monetary value.
- **Custody:** the chain of responsibility for a lot as it passes from cooperative to carrier, terminal and buyer.

CHAPTER TWO: LITERATURE REVIEW

2.1 Introduction

This chapter reviewed the literature on the grand challenge of regional integration and the linked opportunity in agriculture, in order to place the AXK Network project within existing scholarship and practice and to identify the gap it addressed. It covered the causes of low intra-African trade, the institutional nature of agricultural value loss, the use of distributed ledgers to create trust between parties who do not know each other, and the acceptance of new technology by smallholder actors. It then reviewed the systems that already exist, their strengths and weaknesses, and empirical studies of related work. The literature was organised by theme and read critically, with attention to what each source showed and what it left unanswered.

2.2 Conceptual Review

The study rested on a small set of concepts. **Verification** was the ability of a counterparty to confirm a lot's quantity, quality, provenance and custody without relying on the seller's word alone. **Settlement** was the completion of payment for that lot. **Tamper evidence** was the property that any change to a record after it was made would be detectable, which a cryptographic digest provides: if a single character in a record changes, its SHA-256 digest changes completely. **Escrow** described payment held by a neutral party until agreed conditions were met; in this project the neutral party was a smart contract, a program on a public ledger that runs exactly as written and whose state anyone can inspect (Szabo, 1997). **Attestation** was the act of an authorised party committing a record's digest to the ledger at a known time.

The independent variable was the intervention itself: an integrated verification and settlement layer in which an inspection gates payment, and in which each custody event is committed to the ledger as a digest. The dependent variables were the outcomes the study expected the intervention to change: the time needed to complete a trade from agreement to payment, the trust that counterparties placed in records and payment, and their intention to adopt the system. Digital literacy, connectivity and device type were expected to moderate these effects, as the literature on smallholder technology adoption consistently reports (Abebe & Gebresenbet, 2022; Kamilaris et al., 2019). Table 3 summarises the variables.

Table 3: Variables in the study

Type	Variable	How it was observed
Independent	Integrated verification and settlement layer (inspection-gated escrow, attested custody events)	Presence of the AXK workflow in the trade
Dependent	Transaction completion time	Seconds per step on testnet; days from agreement to payment reported by participants
Dependent	Trust in records and payment	Trust items (T1 to T4) in the evaluation survey
Dependent	Intention to adopt	Perceived usefulness, ease of use and behavioural intention items (TAM)
Moderating	Digital literacy, connectivity, device	Participant profile questions; compatibility tests on low-end devices

2.3 Historical Background of the Research Topic

2.3.1 Origins and Early Context of the Research Problem

The problem of trading with a distant stranger is old. Long-distance commodity trade has always depended on intermediaries who vouch for goods and for payment. Merchants relied on agents, bills of lading and, later, bank-issued letters of credit, in which a bank promises to pay the seller when specified documents are presented. These documentary credits were standardised internationally, most recently in the International Chamber of Commerce's UCP 600 rules (International Chamber of Commerce, 2007). They work, but they are paper-heavy, expensive and available mainly to firms with banking relationships.

In African agricultural trade the institutions behind these instruments were thin. Smallholders and small traders mostly traded in cash, through brokers or on personal trust, and the information needed to verify quality and origin rarely travelled with the goods. Studies of postharvest loss showed that much of the value lost between farm and market came from these institutional gaps rather than from spoilage alone (Affognon et al., 2015; Food and Agriculture Organization of the United Nations, 2019). At the same time, trade economists found that Africa's low internal trade was driven more by non-tariff and institutional frictions than by tariffs (Fofack et al., 2021; World Bank, 2020).

2.3.2 Foundational Theories

Two bodies of theory framed the problem from its origins. Transaction cost economics argued that firms and markets exist partly to reduce the costs of searching for partners, negotiating, verifying and enforcing agreements (Coase, 1937; Williamson, 1985). Where verification and enforcement are costly, trade shrinks to partners who already know and trust each other. Institutional economics added that formal and informal rules shape these costs, and that weak institutions make exchange between strangers expensive (North, 1990). Both views predicted what was observed in African agricultural trade: transactions confined to close networks, high risk premiums and value lost to the weakest party.

2.3.3 Key Milestones

Several developments over the last two decades made a new kind of trust infrastructure possible. Table 4 lists the milestones most relevant to this study.

Table 4: Milestones relevant to the study

Year	Milestone	Relevance
1997	Smart contracts described as computer protocols that enforce agreements (Szabo, 1997)	Concept behind the escrow contract
2007	M-Pesa launched in Kenya; GS1 EPCIS 1.0 ratified	Mobile money as Africa's main digital payment rail; a standard for supply-chain events
2008	Bitcoin white paper (Nakamoto, 2008); Open Data Kit started (Hartung et al., 2010)	Public ledgers without a central operator; offline data collection for low-connectivity settings
2014 to 2015	Ethereum smart contract platform (Buterin, 2014); Stellar network and the Stellar Consensus Protocol (Mazières, 2015)	General-purpose smart contracts; a ledger designed for low-cost payments and issued assets
2018	AfCFTA agreement signed in Kigali (African Union, 2018); IBM Food Trust and TradeLens launched	Policy push for intra-African trade; first large blockchain traceability platforms
2021	Trading under the AfCFTA began on 1 January	Market demand for cross-border trade tools
2022	PAPSS commercial launch; GS1 EPCIS 2.0 ratified; TradeLens discontinuation announced	Local-currency settlement between central banks; JSON-based traceability standard; evidence that traceability alone did not sustain a platform
2024	Soroban smart contracts enabled on the Stellar main network (Stellar Development Foundation, 2024)	Smart contracts on a payments-focused ledger, used in this project

2.3.4 Historical Gaps and Transition to the Current Study

The history shows two lines of development that rarely met. Payment innovation, from letters of credit to mobile money and PAPSS, made money move faster and more cheaply but never verified the goods being paid for. Traceability innovation, from EPCIS to blockchain platforms, recorded where goods had been but left payment to separate systems. Large traceability platforms also served mainly multinational supply chains, and at least one, TradeLens, was discontinued after failing to attract enough industry participation (A.P. Moller - Maersk, 2022; Jensen et al., 2019). Smallholder cooperatives and small traders in East Africa were served by neither line. The current study placed itself at the point where the two lines meet: a single workflow in which verification decides whether payment can move.

2.4 Theoretical Review

2.4.1 Foundational Theories

Transaction cost economics. Williamson (1985) identified asset specificity, uncertainty and frequency as the drivers of transaction costs. Cross-border commodity trade

between smallholders and distant buyers scores high on uncertainty: the buyer cannot see the goods and the seller cannot see the money. Kamilaris et al. (2019) described a shared ledger as a digital institution of trust that lowers this uncertainty, and therefore transaction costs, by giving both sides the same tamper-evident record. The theory predicted that if verification and enforcement became cheap and reliable, trade would extend beyond existing networks.

Institutional theory. North (1990) argued that institutions are the rules of the game that reduce uncertainty in exchange. A smart contract can act as a narrow, automated institution: its rules, such as releasing payment only after an inspection pass, are public and enforced without discretion. The literature also warned that the integrity of a ledger record does not guarantee that the information was true when it was captured (Creydt & Fischer, 2019). The institution is only as good as the people and processes that feed it, which is why this project made independent inspection, not self-declaration, the condition for payment.

Technology Acceptance Model. Davis (1989) showed that perceived usefulness and perceived ease of use shape people's intention to use a technology. Venkatesh and Davis (2000) extended the model and confirmed these constructs over time, and Gefen et al. (2003) showed that trust is a further determinant of acceptance in settings where users must rely on a system they cannot fully see into, such as online commerce. For a payment and verification system used by smallholders, trust is central, so this study measured usefulness, ease of use, trust and behavioural intention.

Design science. Hevner et al. (2004) framed information systems research as the building and evaluation of artefacts that solve identified problems, judged by their utility. Peffers et al. (2007) set out a process for it: identify the problem, define objectives, design and develop, demonstrate, evaluate and communicate. This matched the study, which built a working system and evaluated it both technically and with users.

Theoretical framework. Combining these theories, the study treated the AXK layer as an artefact (design science) that acts as an institution of trust (institutional theory and transaction cost economics). Its mechanisms, inspection-gated escrow and attested custody records, were expected to reduce verification and enforcement costs and so shorten transaction time and increase trust. Whether people would adopt it was expected to depend on perceived usefulness, ease of use and trust (TAM). The evaluation in Chapter Four measured the technical mechanisms directly and the adoption constructs through acceptance testing.

2.5 Overview of Existing Systems

Five kinds of existing systems addressed parts of the problem.

PAPSS settles payments between African countries in local currencies through participating central banks and commercial banks, removing the need to route through foreign currencies (Pan-African Payment and Settlement System, 2021). It reduces the cost and time of payment but carries no information about goods, and it is accessed through licensed financial institutions.

Letters of credit and correspondent banking give a seller a bank's promise of payment against documents (International Chamber of Commerce, 2007). They protect both

sides but require banking relationships, collateral and fees that small traders cannot meet, and the documents themselves can be forged or delayed.

Mobile money, led by M-Pesa in Kenya and MTN Mobile Money in Rwanda, lowered the cost of sending money across distance and improved household resilience (Jack & Suri, 2014; Suri & Jack, 2016). It is the most widely used digital payment rail on the continent (Osabutey & Jackson, 2024), but it does not hold funds conditionally or verify goods. Open-source switching platforms such as Mojaloop aim to connect these wallets (Mojaloop Foundation, 2023).

Blockchain traceability platforms such as IBM Food Trust and TradeLens recorded supply-chain events on permissioned ledgers. Walmart's pilot with IBM reduced the time to trace mangoes from about seven days to seconds (Kamath, 2018), which showed the value of shared records. These platforms targeted large firms, did not settle payment, and TradeLens was discontinued in 2023 for lack of industry-wide participation (A.P. Moller - Maersk, 2022). Research prototypes, such as the EPCIS-based traceability system of Lin et al. (2019), showed that standards and ledgers can be combined, but they stopped at recording.

Field data tools such as Open Data Kit (ODK) let inspectors and enumerators collect structured data offline on phones and synchronise later (Hartung et al., 2010). They are widely used in agriculture and health in low-connectivity settings, but they produce data, not trust: an ODK record can be edited before it reaches a buyer.

2.5.1 Strengths and Weaknesses of the Existing Systems

Table 5: Strengths and weaknesses of existing systems

System	Strengths	Weaknesses for this problem
PAPSS	Real-time, local-currency settlement; backed by central banks	No verification of goods; accessed only through licensed institutions
Letters of credit	Strong payment assurance; international rules	Slow, costly, document-based; out of reach for smallholders
Mobile money (M-Pesa, MoMo)	Very wide reach; low cost; familiar to users	No conditional holding of funds; no link to goods
IBM Food Trust, TradeLens	Shared, tamper-evident records; proven tracing speed	Built for large firms; no settlement; TradeLens discontinued
ODK and similar tools	Offline capture; open source; proven in the field	Records can be changed before sharing; no settlement
AXK Network (this study)	Inspection gates payment; custody events attested; public verification; open source	Testnet only; custodial keys; pilot evidence still limited

2.6 Review of Related Works (Empirical Review)

Empirical studies informed both the design and the evaluation. Jack and Suri (2014) used household panel data from Kenya and found that mobile money users absorbed income shocks without reducing consumption, because the cost of sending money had fallen; Suri and Jack (2016) later linked access to mobile money to a measurable fall in poverty,

particularly among female-headed households. These studies showed that lowering the cost of moving value changes economic behaviour, which supported the settlement half of this project.

Affognon et al. (2015) carried out a meta-analysis of postharvest loss studies in sub-Saharan Africa and found that losses were large, inconsistently measured and poorly addressed by interventions focused on a single stage of the chain. They called for approaches that span the whole chain, which this project attempted by covering custody from registration to settlement. Abebe and Gebresenbet (2022) reviewed digital agriculture in East Africa and found that digital tools improved market access but were limited by infrastructure and digital literacy, which shaped this project's emphasis on low-end devices and offline inspection.

On the ledger side, Kamilaris et al. (2019) reviewed blockchain projects in agriculture and food and found most were at pilot stage, with a recurring difficulty in getting smallholders to enter accurate data. Saberi et al. (2019) and Kouhizadeh et al. (2021) studied adoption barriers and found that organisational, supply-chain and external barriers mattered as much as technical ones. Casino et al. (2019) systematically reviewed blockchain applications and cautioned that the technology should be chosen for the requirements of a use case, not assumed. Kshetri (2018) linked blockchain to supply-chain objectives such as cost, quality and risk reduction, and Galvez et al. (2018) examined its use for food traceability analysis. Creydt and Fischer (2019) stressed that a ledger cannot make a false input true, and Lin et al. (2019) built a traceability system combining blockchain with EPCIS. Mavilia and Pisani (2022) studied blockchain in South African agriculture and noted the gap between pilot enthusiasm and scaled use.

2.6.1 Summary of Reviewed Literature

Table 6: Summary of reviewed empirical literature

Author(s)	Focus and method	Key finding	Gap relevant to this study
Jack and Suri (2014)	Mobile money in Kenya; household panel	Lower transfer costs improved risk sharing	No link to goods or trade verification
Suri and Jack (2016)	Long-run effects of mobile money; panel	Access reduced poverty	Payment only
Affognon et al. (2015)	Postharvest loss; meta-analysis	Losses large and institutional; whole-chain approaches needed	No integrated digital solution tested
Abebe and Gebresenbet (2022)	Digital agriculture in East Africa; review	Tools help market access; limited by literacy and infrastructure	Few working systems evaluated
Kamilaris et al. (2019)	Blockchain in agri-food; review	Ledger as institution of trust; data entry is a weakness	Settlement not addressed
Saberi et al. (2019); Kouhizadeh et al. (2021)	Adoption barriers; conceptual and survey	Organisational and external barriers are decisive	No East African smallholder evidence
Lin et al. (2019)	Blockchain with EPCIS; system build	Standards and ledgers can be combined	Recording only; no payment
Kamath (2018)	Walmart and IBM pilots; case study	Tracing time cut from days to seconds	Large retailer, permissioned ledger, no settlement
Creydt and Fischer (2019)	Food traceability; review	Ledger integrity does not prove input truth	Need for independent verification at capture
Mavilia and Pisani (2022)	Blockchain in South African agriculture; case review	Pilots rarely scale	Few results from cooperative-level trade

2.7 General Comments

The literature agreed that regional trade in Africa is held back by institutional and informational frictions, that much agricultural value loss is institutional, and that shared ledgers are a credible way to reduce verification costs. It also agreed that technology alone is not enough: inputs must be trustworthy, users must find the system useful, easy and trustworthy, and adoption barriers are organisational as well as technical. What it lacked was a system that joins commodity verification and settlement in one workflow for smallholder actors, together with evidence from an East African setting. This study responded by building such a system around an independent inspection that decides whether payment can move, by recording every custody step on a public ledger, and by evaluating it both technically and with prospective users.

CHAPTER THREE: SYSTEM ANALYSIS AND DESIGN

3.1 Introduction

This chapter described how the AXK Network system was designed and how it was to be evaluated. It set out the research design and development model, the data the evaluation drew on, the baseline and proposed models, and the metrics used to judge the result. It then presented the functional and non-functional requirements, the design diagrams (architecture, use cases, data model, classes, sequence and state flow) and the tools used to build the system.

3.2 Research Design

The study followed a design science approach (Hevner et al., 2004; Peffers et al., 2007): it identified a problem, built an artefact to address it, demonstrated the artefact in use and evaluated it. Evaluation used mixed methods. The quantitative strand measured objective system behaviour, such as test outcomes, response times and the time taken by each step of a trade on the Stellar testnet, together with survey scores on the Technology Acceptance Model constructs. The qualitative strand used open survey answers and optional interviews to explain those scores.

The software was built with an iterative and incremental development model (Larman & Basili, 2003), shown in Figure 1. Each iteration moved through requirements, design, build, testing and evaluation, and its results fed the next. This suited a project whose requirements were expected to change through contact with cooperatives, inspectors and buyers, and it is what happened in practice: the logistics and port steps, the offline inspection import and the redesign of the screens around a custody record were all added in later iterations in response to what earlier ones revealed.

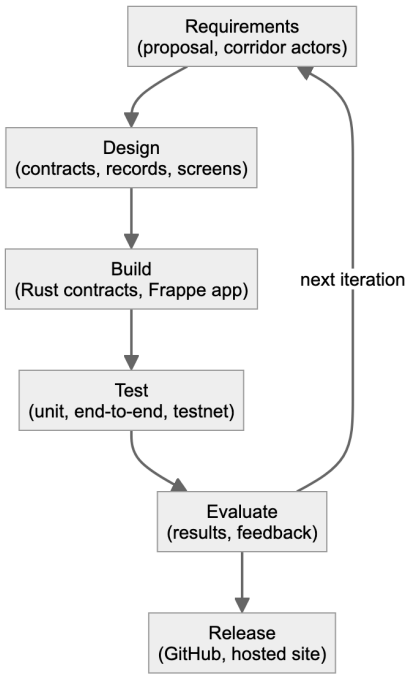


Figure 1: Iterative and incremental development model used in the project

Acceptance testing followed the plan in the proposal. Participants were adults with current or recent experience of agricultural trading, cooperative work, buying or logistics, recruited purposively through cooperative and trader networks connected to Kigali; one was a student with agribusiness experience who watched the full demonstration. The other participants each used the role screen that matched their work on the hosted system, using a demo account, and every participant then completed an online questionnaire. The questionnaire collected a short profile, four items each for perceived usefulness, perceived ease of use and trust, three items for behavioural intention, all on a seven-point agreement scale, the number of days between agreeing their last cross-border deal and receiving payment, and open questions on what was useful and what would stop them using the system. Construct scores were summarised as means and standard deviations, internal consistency was checked with Cronbach's alpha (Cronbach, 1951), and open answers were grouped into themes. The Technology Acceptance Model was used rather than the System Usability Scale (Brooke, 1996) because the research questions concerned trust and adoption as well as usability.

Participation was voluntary and based on informed consent, with the right to withdraw at any time. Participants were identified only by a code (AXK1, AXK2 and so on), no real money was used, and data were stored with restricted access in line with Rwanda's law on the protection of personal data and privacy (Republic of Rwanda, 2021). Ethical approval was granted by the ALU Senate Research Ethics Committee on 2 November 2026 (approval code SEPT26-BSE-039), and all acceptance testing took place after that date.

3.2.1 Data Sources

The project was not in the machine learning track, so no training dataset was used. The evaluation drew on four sources: the transaction records of every trade run on the Stellar

testnet, stored with their transaction hashes in the application database and verifiable on a public block explorer; the output of the automated test suites; the measurements from the compatibility and performance scripts; and the responses from acceptance testing. A synthetic five-year history of lots and events, clearly labelled as synthetic, was used only to populate the historical view of the chain explorer and was not used in any result.

3.2.2 Baseline Model Description

The baseline was current practice as described in Chapter Two and by participants. A cooperative and a cross-border buyer agree a deal by phone or in person. Quality is checked, if at all, by the buyer's agent or by personal trust, and the result is not recorded anywhere the other party can independently check. Payment is made in advance, on delivery in cash or by mobile money, or, for formal firms, through a bank. Custody between the cooperative and the destination is evidenced by paper waybills. Whoever pays or ships first carries the risk, disputes are settled by negotiation, and the time from agreement to payment is measured in days or weeks.

3.2.3 Proposed Model Description

In the proposed model, each step of the trade is a signed transaction on a public ledger, and payment depends on verification. A cooperative registers a lot, and the lot record's digest is committed to the lot registry contract. An inspector on the registry's allow-list passes or rejects the lot, and the inspection report's digest is recorded. A buyer can fund an escrow only for a lot that has passed; the escrow contract asks the registry whether the lot passed and who owns it, so payment cannot be redirected. A carrier dispatches the lot under a container seal and the waybill is attested. At the port or inland terminal, the arriving seal is compared with the waybill; a mismatch raises a tamper alert and nothing is recorded. When the buyer confirms delivery, the escrow releases payment, the splitter divides it between the cooperative and its members, and a final settlement record linking every earlier digest is attested. Anyone can then verify the lot from a public page or QR code.

3.2.4 Metrics for Evaluation

Table 7: Evaluation metrics and targets

Metric	Target	Method
Correctness of contract rules	All tests pass; every unauthorised or out-of-order action refused	Contract unit tests; failure-mode tests; testnet runs
Integrity of records	Every stored record matches its on-chain digest	Chain verification on settled lots
Trade completion time on testnet	Under 3 minutes from registration to settlement	Timed full trade; database timestamps of settled lots
Page response	Median screen ready under 1 s on a normal connection; under 5 s worst case on a slow 3G phone	Playwright compatibility runs
Compatibility	All screens render in Chromium, Firefox and WebKit at phone, tablet and desktop sizes with no sideways overflow	Playwright compatibility runs
Acceptance	Construct means above the scale midpoint (4 of 7); Cronbach's alpha of at least 0.7	Acceptance testing questionnaire

3.3 System Analysis

3.3.1 Functional Requirements of the System

Table 8: Functional requirements

ID	Requirement	Actor
FR1	Users sign in with a demo account for each role, and guests cannot sign trade actions. On the hosted demo, any signed-in demo account can perform any role's action, so roles are not yet enforced per user	All
FR2	Register a lot (commodity, variety, grade, quantity, origin, harvest date) and commit its digest on-chain	Cooperative
FR3	Pass or reject a lot with grade, measured quantity and moisture; a rejection requires a reason; import inspections captured offline	Inspector
FR4	Fund an escrow only for a lot that has passed inspection, with a refund deadline	Buyer
FR5	Record dispatch with carrier, vehicle, container seal and destination, and attest the waybill	Logistics carrier
FR6	Record gate-in at a port or terminal, check the arriving seal against the waybill and attest the receipt	Port / terminal
FR7	Confirm delivery: release escrow, split proceeds between cooperative and members, attest the settlement record	Buyer
FR8	View any lot's public history and custody timeline by reference or QR code, without an account	Public verifier
FR9	Re-check stored records against the on-chain digests	Public verifier
FR10	Export a lot's history as a GS1 EPCIS 2.0 document	Public verifier
FR11	Explore live contract events and the flow of funds	All

3.3.2 Non-functional Requirements of the System

Table 9: Non-functional requirements

Category	Requirement
Security	Contract rules enforce who may act and in what order; signing keys never reach the browser; signing endpoints require a role and are rate-limited
Integrity	Records are hashed in a canonical form; only digests go on-chain; any change is detectable
Privacy	Public pages show no personal data about farmers, inspectors or buyers
Usability	Plain labels, one task per screen, clear error messages; suitable for users with varying digital literacy
Performance	Screens open quickly on low-end phones and slow connections
Availability	Offline inspection capture where connectivity is poor; clear errors when the network is slow
Compatibility	Current Chrome, Edge, Firefox and Safari on phone, tablet and desktop
Maintainability	Object-oriented ledger package, documented contracts, automated tests, one command per task
Extensibility	New corridors, commodities and payout rails can be added without changing the contracts' core rules
Openness	Source code released under the MIT licence with contribution guidelines

3.3.3 Proposed Model Diagram

Figure 2 shows the proposed model as the life cycle of a lot. Each arrow is a signed transaction; the state machine in the application refuses moves that skip a step, and the contracts refuse them again on-chain.

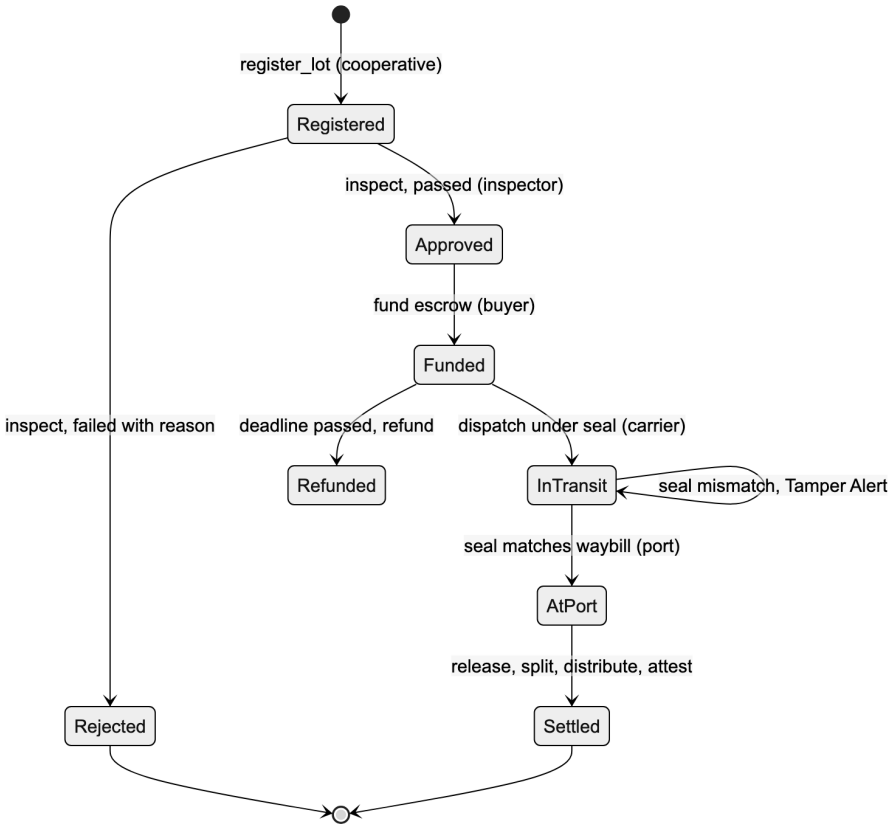


Figure 2: Life cycle of a lot in the proposed model

3.4 System Design

3.4.1 Architecture Diagram

The system followed a layered architecture (Figure 3). The presentation layer contained the role screens, the public pages and the ODK form for offline inspection. The application layer, a Frappe v15 application written in Python, exposed whitelisted endpoints that check the user's role and rate limit, and passed each trade step to the ledger package. The data and trust layer combined a MariaDB database for full records and application state with four Soroban contracts on the Stellar testnet for tamper-evident commitments and payment. This kept the trust-critical rules in the contracts, independent of the user interface, and treated the ledger as one component of the system rather than a replacement for it (Cole et al., 2019).

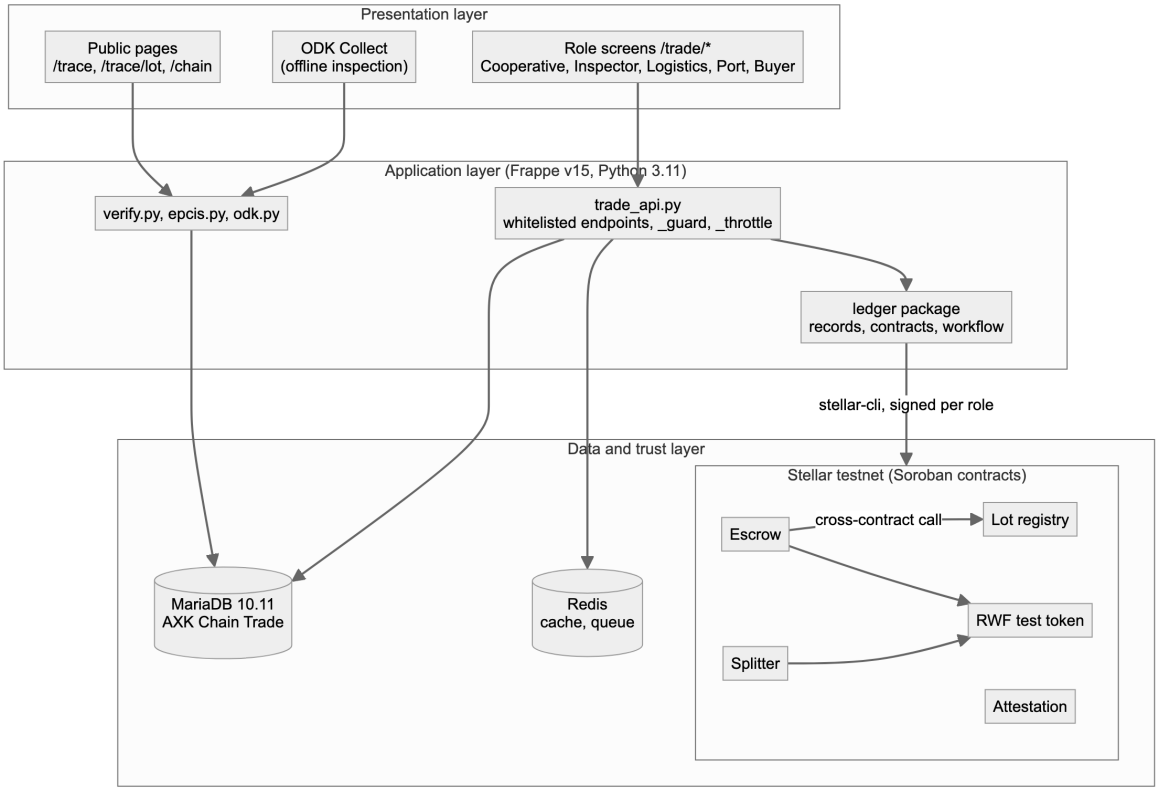


Figure 3: System architecture of AXK Network

3.4.2 Use Case Diagram

Figure 4 shows the actors and their use cases. Each trade role has its own screen. The public verifier needs no account, which matters because a buyer's bank, a customs officer or a regulator may need to check a lot without joining the system.

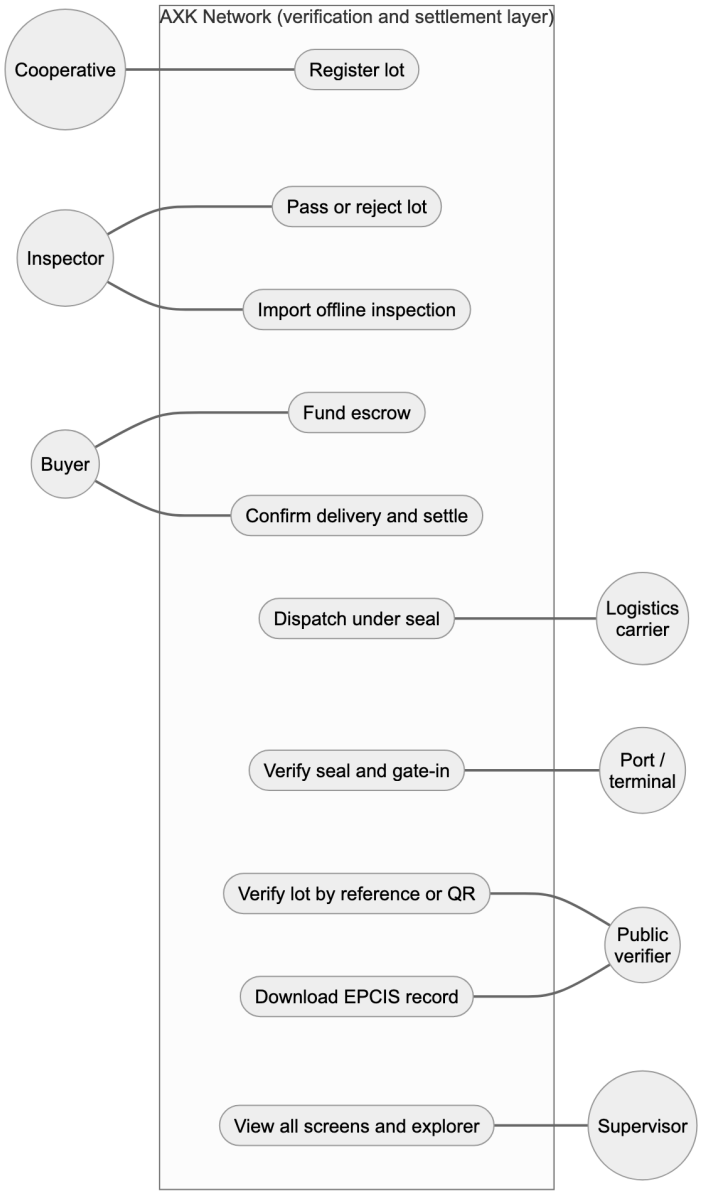


Figure 4: Use case diagram

3.4.3 Database Design

Figure 5 shows the data model. The `AXK Chain Trade` document in MariaDB holds the full record of each lot: the business data, the canonical records and their digests, and one transaction hash per on-chain step. The contracts hold only what is needed to enforce the rules and prove the records: the lot's owner, digest and status in the registry, the inspection result, the escrow terms, the split shares and the attestations. The lot identifier is derived on-chain as a hash of the cooperative's address and the lot reference, so no one can register a lot in another cooperative's name.

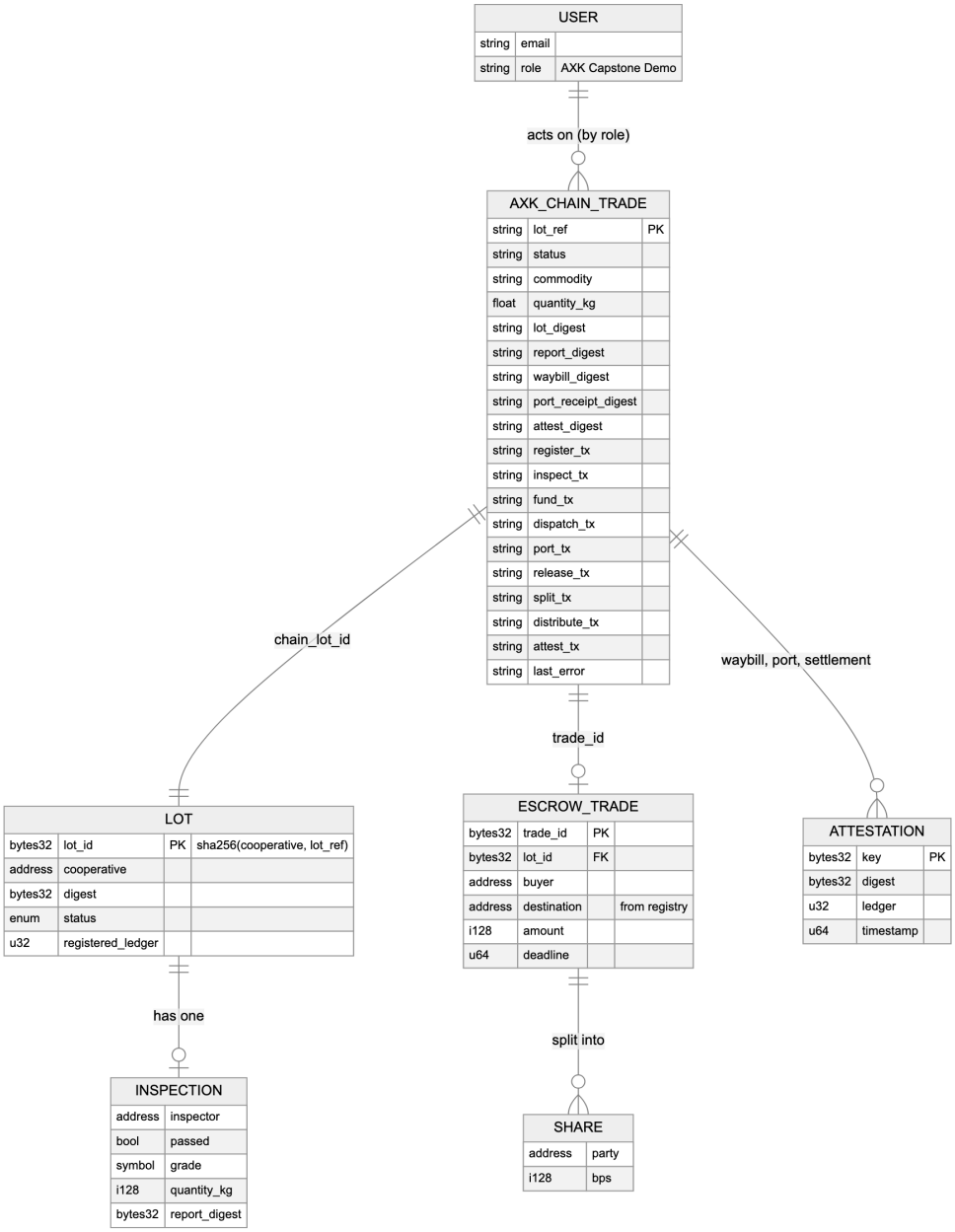


Figure 5: Entity relationship diagram (off-chain document and on-chain records)

3.4.4 Class Diagram

The ledger package was designed with object-oriented principles (Figure 6). Abstract record classes define how any record is made canonical and hashed, and subclasses describe each record type; attested records add their own attestation key. A contract client hides how a call is built, signed and decoded, and one subclass per contract adds that contract's functions and error table. The command runner abstraction allowed the real Stellar command-line tool to be replaced by a fake in tests. `TradeWorkflow` coordinates the steps and enforces their order.

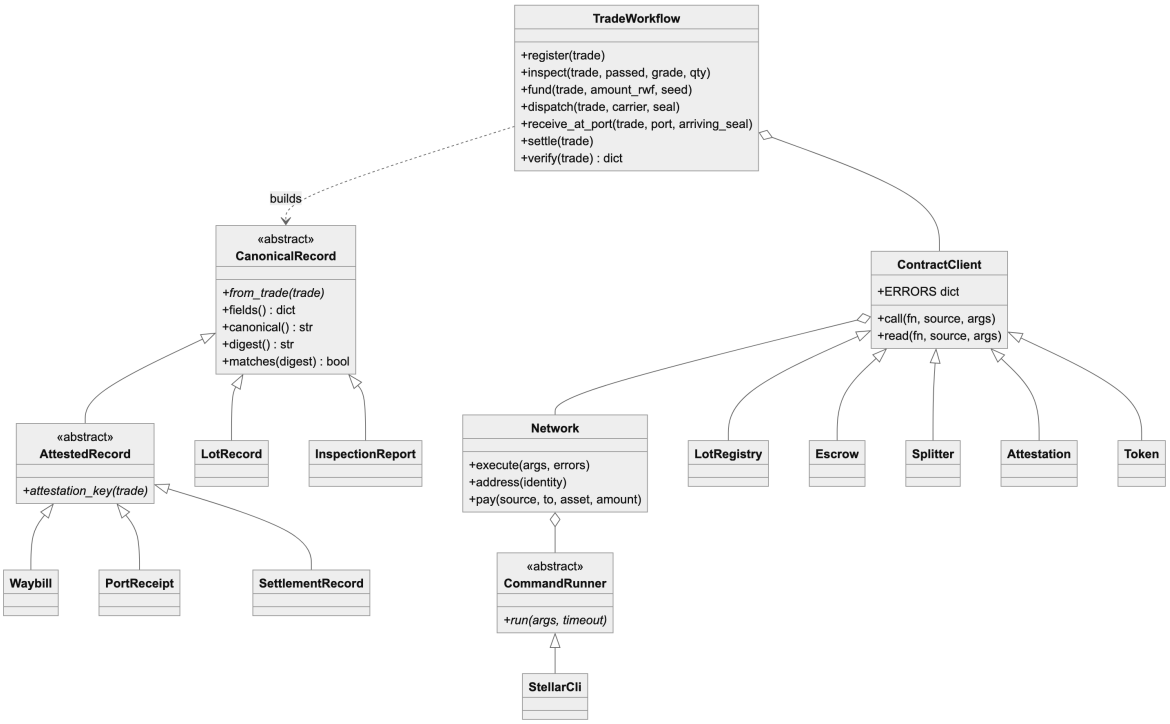


Figure 6: Class diagram of the ledger package

3.4.5 Sequence Diagram

Figure 7 shows the most important interaction, funding the escrow. The workflow first checks that the buyer can pay, then the escrow contract asks the lot registry whether the lot has passed inspection and who should be paid. Only if the registry names the cooperative does the escrow take the buyer's tokens. The buyer never supplies the payee, which removes a whole class of fraud.

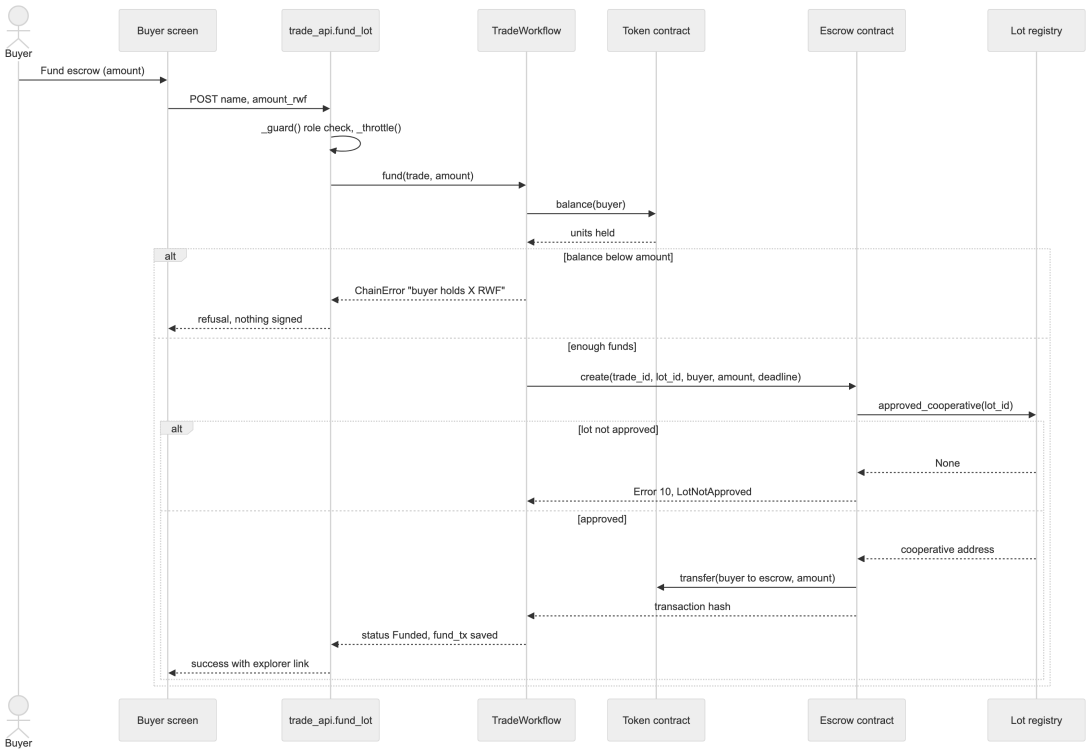


Figure 7: Sequence diagram for funding an escrow

3.5 Development Tools

Table 10: Development tools and versions

Tool	Version	Use
Rust with soroban-sdk	Rust 1.98, soroban-sdk 22.0.11, target wasm32v1-none	Smart contracts
Stellar CLI	28.0.0	Building, deploying, signing and invoking contracts
Stellar testnet	Soroban RPC and Horizon	Contracts, test token, transactions
Python	3.11	Application, scripts and tests
Frappe framework and bench	15.104.0 and 5.31.0	Application server, roles, REST API, web pages
MariaDB	10.11	Application database
Redis	7 or later (tested 8.10)	Cache and job queue
HTML, CSS, JavaScript (no framework)	ES2020, WebCrypto	Role screens, public pages, in-browser re-hashing
ODK Collect and pyxform	Current release	Offline inspection form
Playwright	Chromium 156, Firefox 157, WebKit 27.2	Compatibility and screenshot testing
Git and GitHub		Version control and public repository
Ubuntu, nginx, supervisor	Ubuntu 24.04	Hosted deployment at capstone.axk.org

The tools differed from those named in the proposal, which listed React, Node.js and PostgreSQL. Frappe was chosen during the first iteration because it provided authentication, roles, a REST API, a database layer and web pages out of the box, and because it matched the platform used by AXK Network, which left more of the limited time for the smart contracts. Plain HTML, CSS and JavaScript were used for the screens to keep pages small for low-end phones.

CHAPTER FOUR: SYSTEM IMPLEMENTATION, RESULTS AND DISCUSSION OF FINDINGS

4.1 System Implementation

4.1.1 Introduction

This section reported how the design in Chapter Three was turned into working software: the smart contracts, the ledger package that connects the application to them, the access controls, and the hosted deployment. Short extracts of the source code are included for the main modules. The full code is in the public repository at github.com/tejirijesse/axk-open-source-capstone.

4.1.2 Smart Contracts

Four contracts were written in Rust with the Soroban SDK and deployed to the Stellar testnet: the lot registry, the escrow, the splitter and the attestation registry. Each contract keeps its state private and changes it only through functions that check who is calling. Failures are returned as typed errors rather than panics, so the application can show a clear message. Listing 1 shows how the registry derives the lot identifier from the cooperative's address and the lot reference, so a lot cannot be registered twice or in another cooperative's name.

Listing 1: Registering a lot in the lot registry contract (contracts/lot_registry/src/lib.rs)

```
pub fn register_lot(env: Env, cooperative: Address, lot_ref: BytesN<32>,
                    digest: BytesN<32>) -> Result<BytesN<32>, Error> {
    cooperative.require_auth();
    let id = Self::lot_id(env.clone(), cooperative.clone(), lot_ref.clone());
    let key = Key::Lot(id.clone());
    if env.storage().persistent().has(&key) {
        return Err(Error::LotExists);           // write-once
    }
    env.storage().persistent().set(&key, &Lot { cooperative: cooperative.clone(),
        lot_ref, digest, status: LotStatus::Registered,
        registered_ledger: env.ledger().sequence() });
    env.events().publish((symbol_short!("lot"), symbol_short!("register")),
(id.clone(), cooperative));
    Ok(id)
}
```

The escrow is the core of the design (Listing 2). Before taking any money, it asks the registry for the cooperative that owns an approved lot. If the lot has not passed inspection, the call fails with error 10 and nothing moves. The payee is never an argument, so a buyer or an attacker cannot redirect the payment. The contract records its state before transferring tokens, which follows the checks, effects and interactions pattern.

Listing 2: Inspection-gated escrow creation (contracts/escrow/src/lib.rs, shortened)

```
pub fn create(env: Env, trade_id: BytesN<32>, lot_id: BytesN<32>, buyer: Address,
              token_id: Address, amount: i128, deadline: u64) -> Result<(), Error> {
    if amount <= 0 { return Err(Error::AmountNotPositive); }
    // ... deadline checks, one trade per id, one live escrow per lot ...
    let destination = RegistryClient::new(&env, &Self::registry(env.clone())?)
        .approved_cooperative(&lot_id)
        .ok_or(Error::LotNotApproved)?;         // payment only for passed lots
    buyer.require_auth();
    Self::verifier(env.clone())?.require_auth();
    env.storage().persistent().set(&key, &Trade { lot_id, buyer: buyer.clone(),
        destination, token: token_id.clone(), amount, deadline, status:
Status::Funded });
    token::Client::new(&env, &token_id).transfer(&buyer,
&env.current_contract_address(), &amount);
    Ok(())
}
```

The splitter divides a payment between the cooperative and its members in basis points (Listing 3). Integer division truncates, so the last share receives whatever remains; the parts therefore always add up to exactly the amount paid in, and no unit is lost to rounding.

Listing 3: Lossless split of proceeds (contracts/splitter/src/lib.rs)

```
let last = shares.len() - 1;
let mut paid: i128 = 0;
for (i, share) in shares.iter().enumerate() {
    let part = if i as u32 == last { amount - paid }      // remainder to the last
share
               else { amount * share.bps / BPS };
    paid += part;
    if part > 0 { client.transfer(&from, &share.party, &part); }
}
```

4.1.3 Ledger Package and Application Layer

The application was built as a Frappe v15 app. Its blockchain logic was placed in a separate `ledger` package with no Frappe dependency, which made it testable without a server or network. Every record a participant commits to is written as canonical JSON and hashed with SHA-256 (Listing 4); the record stays in the database and only the digest goes on-chain. The same function is used by the browser re-check and by the EPCIS export, so a record always produces the same digest wherever it is computed.

Listing 4: Canonical record and digest (ledger/records.py, shortened)

```
def canonical(obj):
    """Deterministic JSON: the same fields always produce the same bytes."""
    return json.dumps(obj, sort_keys=True, separators=(",", ":"), ensure_ascii=False)

class CanonicalRecord(ABC):
    @classmethod
    @abstractmethod
    def from_trade(cls, trade, **context): ...
    def canonical(self):
        return canonical(self._fields)
    def digest(self):
        return sha256_hex(self.canonical())

class AttestedRecord(CanonicalRecord):
    @staticmethod
    @abstractmethod
    def attestation_key(trade): ...     # Waybill, PortReceipt, SettlementRecord
override this
```

Access to the signing endpoints is controlled in `trade_api.py` (Listing 5). A request must come from a signed-in user with the demo role or a system manager; demo mode without login works only for requests made directly on the server itself, never through a proxy. Each user is limited to 30 signed transactions per hour so that a shared reviewer account cannot be used to flood the testnet.

Listing 5: Role check and rate limit on signing endpoints (trade_api.py)

```
def _guard():
    if frappe.session.user != "Guest" and ({"System Manager", DEMO_ROLE} &
set(frappe.get_roles())):
        return
    if cint(frappe.conf.get("axk_demo_mode")) and _direct_local_request():
        return
    frappe.throw(SIGN_IN, frappe.PermissionError)

def _throttle():
    key = cache.make_key(f"axk_capstone:signs:{frappe.session.user}")
    count = cache.incr(key)
    if count == 1:
        cache.expire(key, 60 * 60)
    if count > SIGN_LIMIT_PER_HOUR:
        frappe.throw("Too many transactions this hour. Try again later.",
frappe.RateLimitExceededError)
```

4.1.4 Deployment

The system ran in three forms. The full stack was hosted at capstone.axk.org on an Ubuntu 24.04 server, with nginx and a Let's Encrypt certificate in front of the Frappe application, MariaDB and Redis, managed by supervisor, and signing to the Stellar testnet. Signing keys were stored on the server under the service user with file permissions restricted to that user and were never placed in the repository or sent to the browser. The same code ran locally for development. A Python-only offline simulation, clearly labelled as not on-chain, was deployed to Vercel so that the screens could be tried without any installation.

4.2 Graphical View of the Project

4.2.1 Screenshots with Description

The screenshots below were taken from the full stack running against the Stellar testnet on 31 October 2026. Each shows a feature from the functional requirements in Table 8 and the action a user can take to address the problem stated in Chapter One.

Sign in with role demo accounts (FR1). Figure 8 shows the sign-in page. Each row is a demo account for one role; clicking a row fills the email and password, and signing in opens that role's screen. The accounts cannot change the shared password, rename themselves or share records, so one tester cannot lock others out.

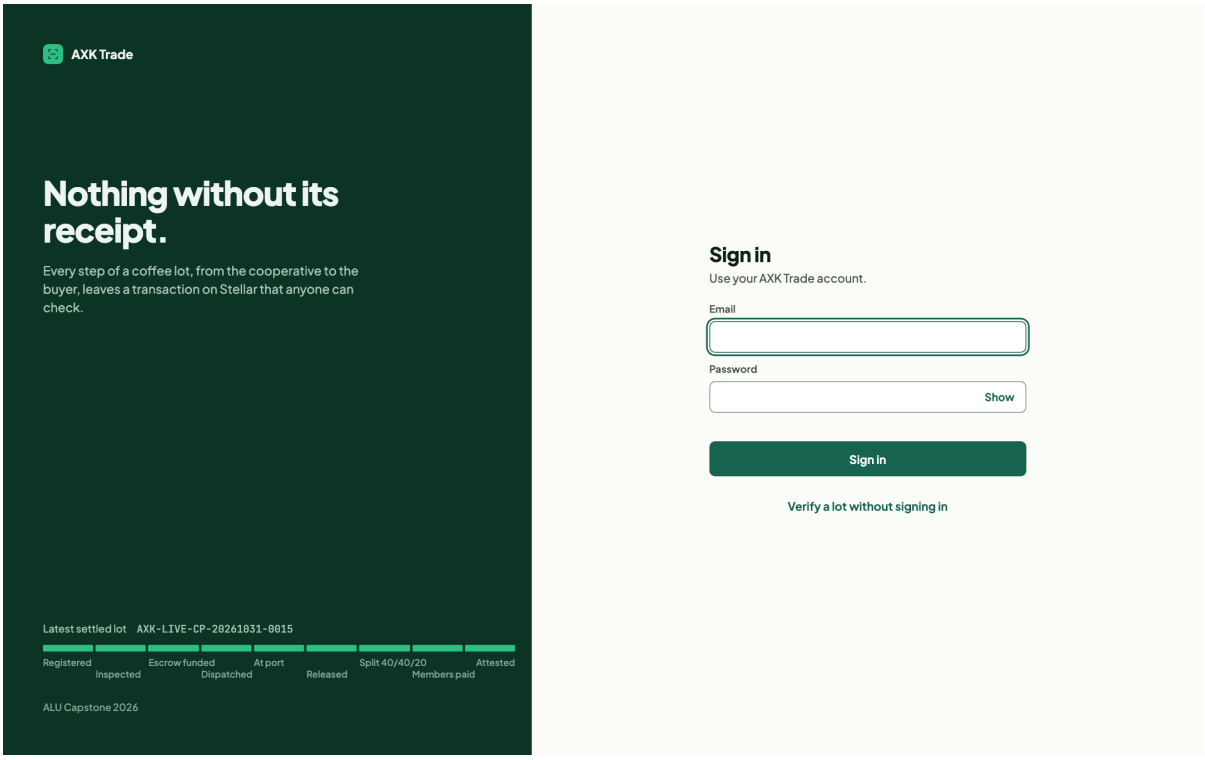


Figure 8: Sign-in page with one demo account per role

Overview (FR1, FR11). Figure 9 shows the overview a supervisor sees: counts of lots at each stage, the work waiting for each role, and recent lots with their custody progress (for example 9/9 for a settled lot) and current status.

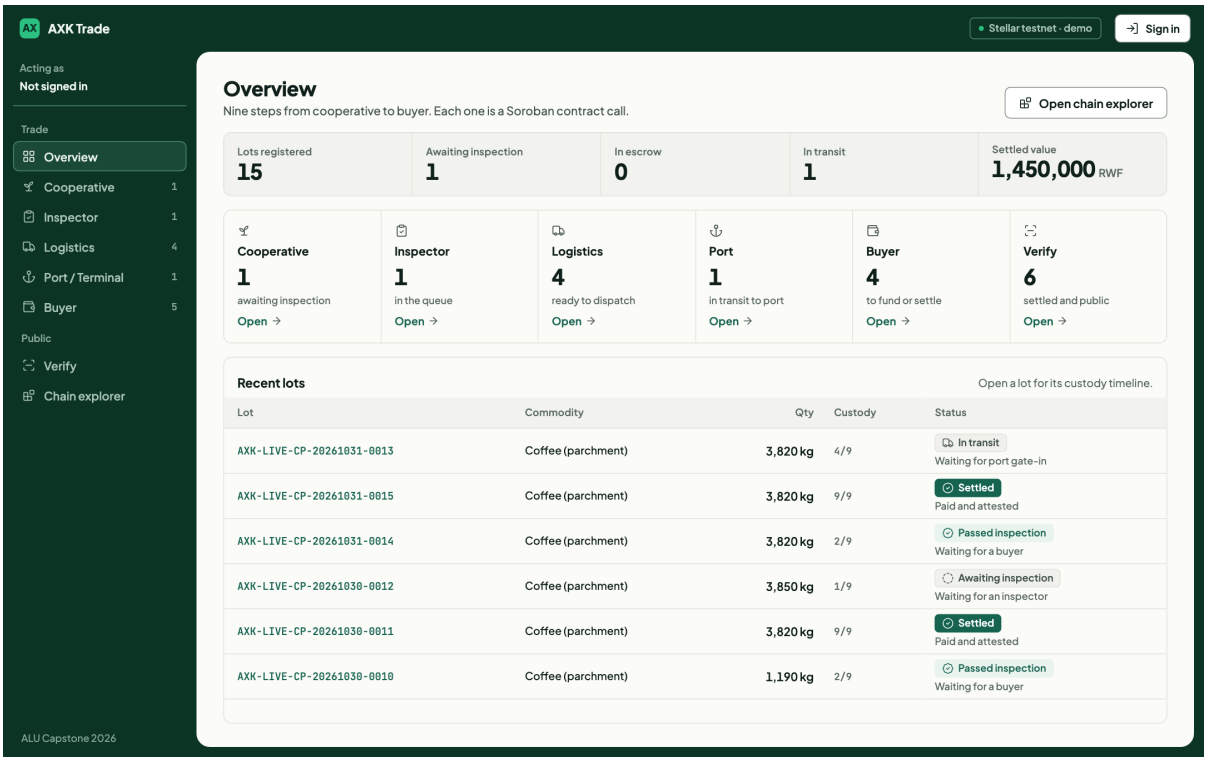


Figure 9: Overview of lots, work per role and custody progress

Register a lot (FR2). On the cooperative screen (Figure 10) the cooperative enters the commodity, variety, grade, quantity, origin and harvest date. On submission the lot record is hashed (Listing 4) and the digest is committed to the lot registry (Listing 1). The new lot appears in the inspector's queue with a link to its transaction.

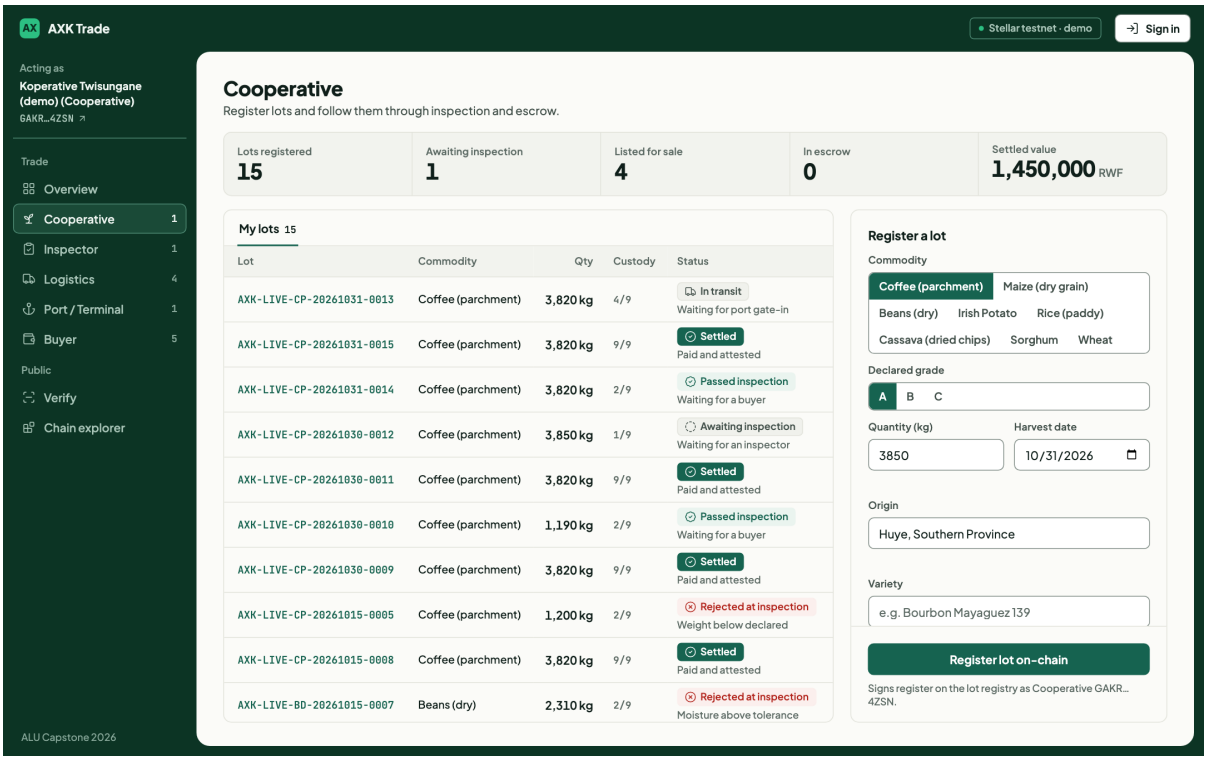


Figure 10: Cooperative screen for registering a lot

Inspect a lot (FR3). The inspector (Figure 11) records the measured grade, quantity and moisture and passes or rejects the lot. A rejection requires a structured reason, which makes rejections comparable across inspectors. Inspections captured offline in ODK can be imported from the third tab.

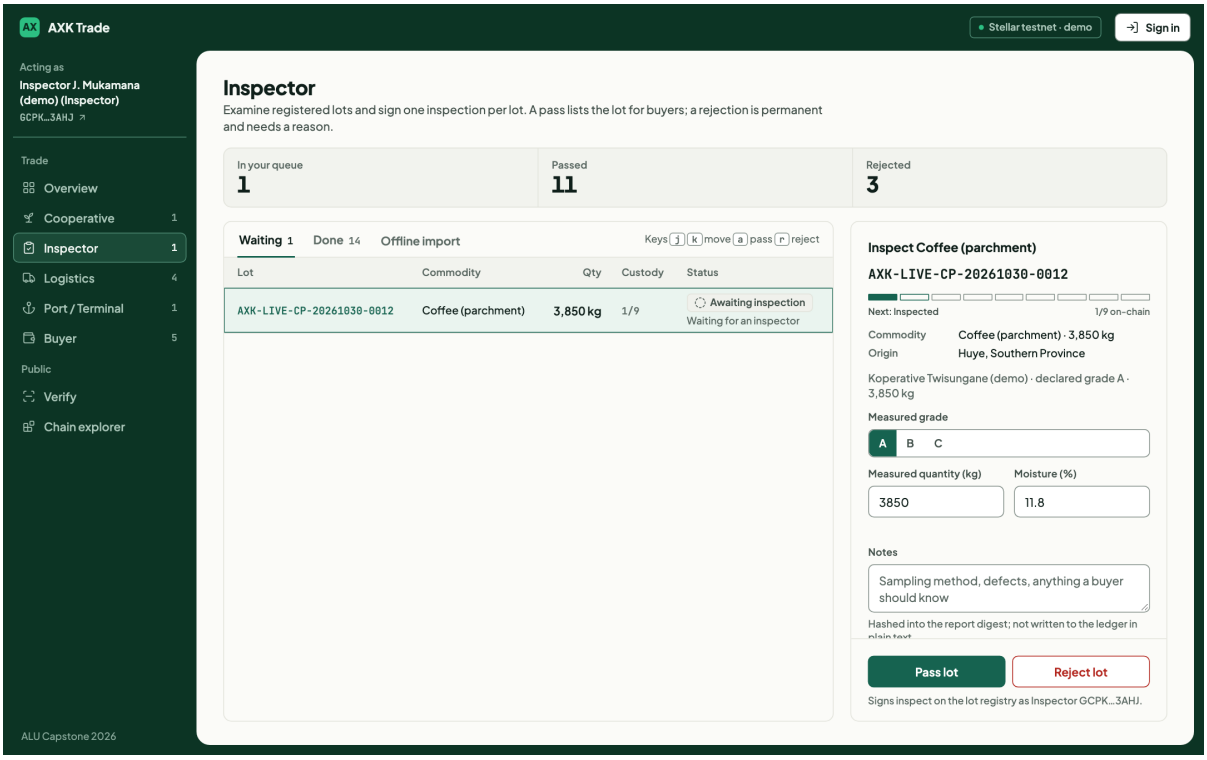


Figure 11: Inspector screen with measured values and pass or reject

Fund an escrow (FR4). The buyer (Figure 12) sees only lots that have passed inspection, with the inspector's measurements. Funding locks the payment in the escrow contract, which checks the lot's approval with the registry before taking the tokens (Listing 2 and Figure 7).

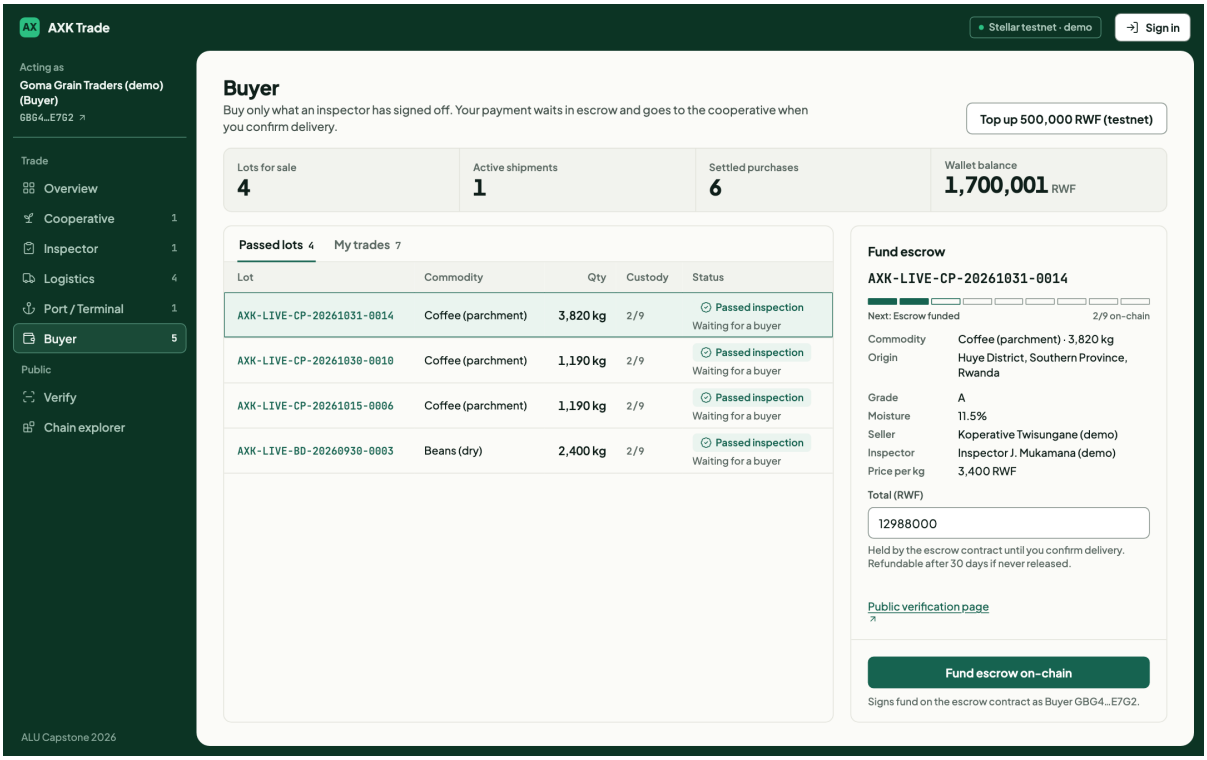


Figure 12: Buyer screen for funding an escrow on an inspected lot

Dispatch under seal (FR5). The logistics carrier (Figure 13) records the carrier, driver, vehicle, container seal and destination for a funded lot. The waybill is hashed and attested on-chain.

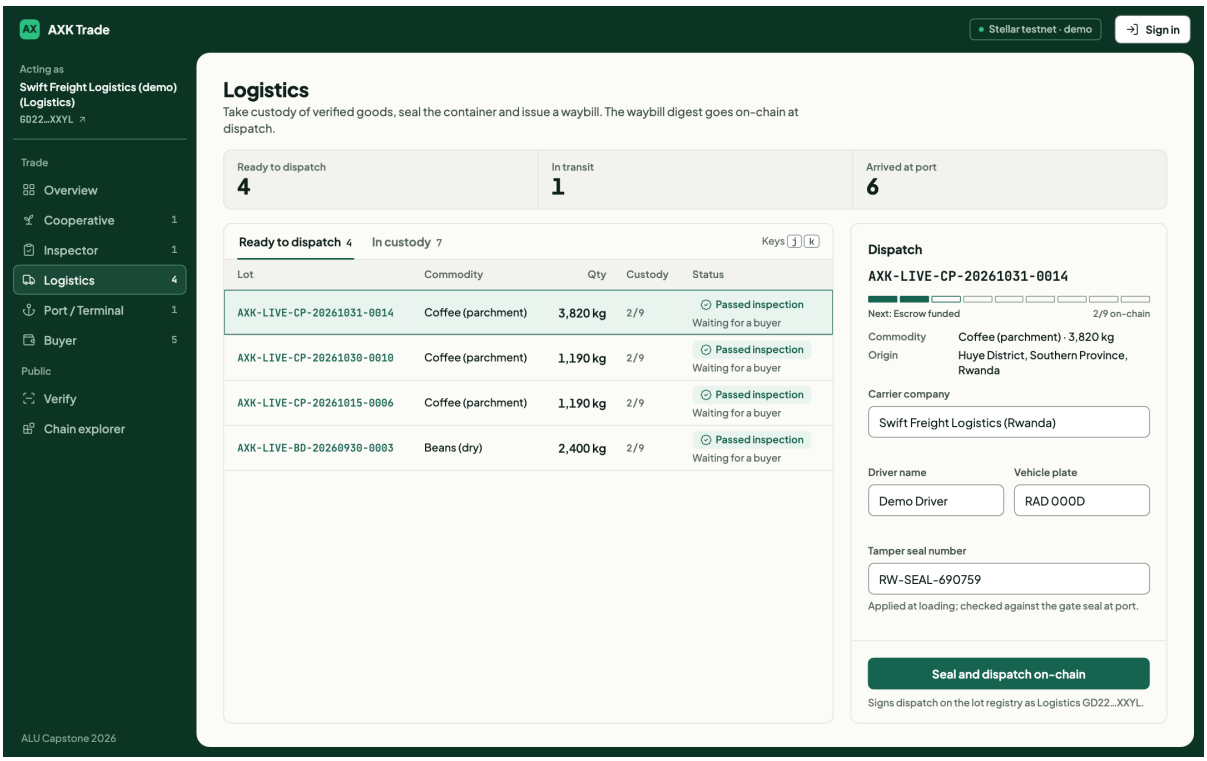


Figure 13: Logistics screen for dispatching a funded lot under a container seal

Gate-in with seal check (FR6). At the port or inland terminal (Figure 14), the operator enters the container, customs reference and the seal number found on arrival. The system compares it with the seal on the waybill before anything is signed.

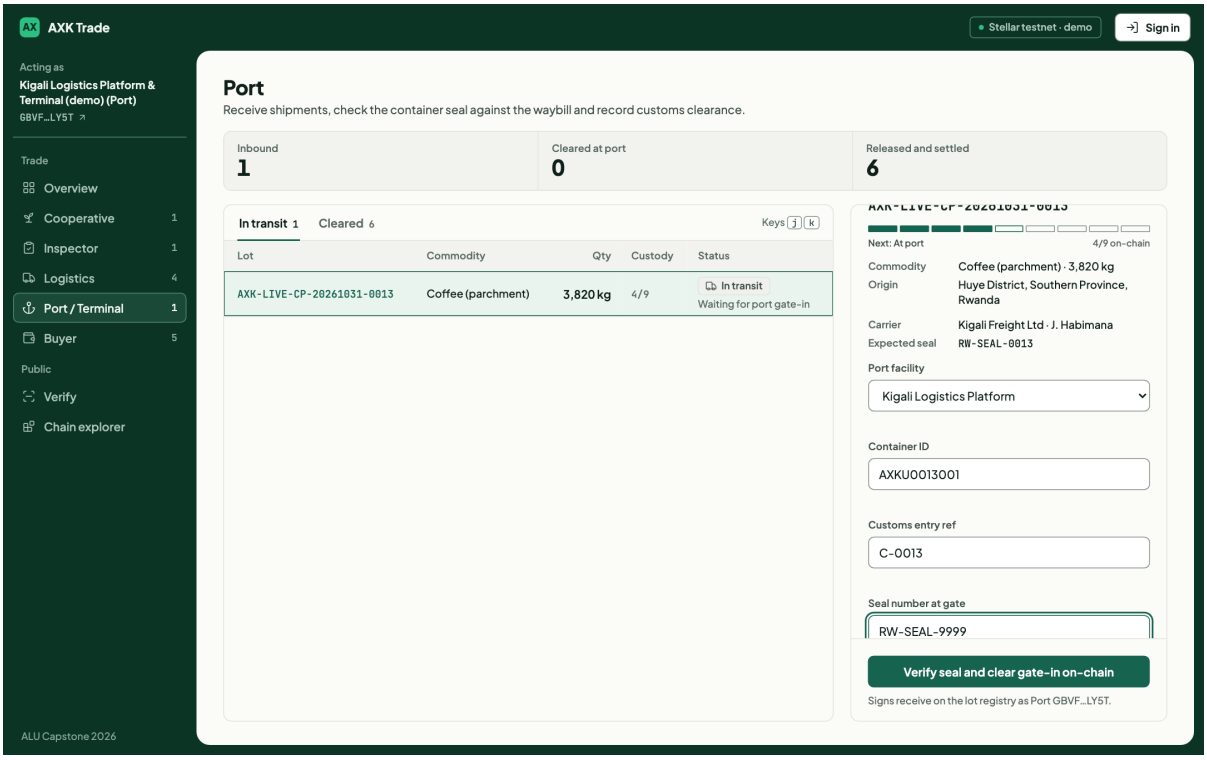


Figure 14: Port screen showing the expected seal and the gate-in form

Custody timeline (FR7, FR9). Figure 15 shows a settled lot. The Custody Strip has nine notches, one per on-chain step, and a notch fills only when its step has a real transaction hash. Each step links to its transaction on the Stellar Expert explorer, and the records can be re-checked against the chain.

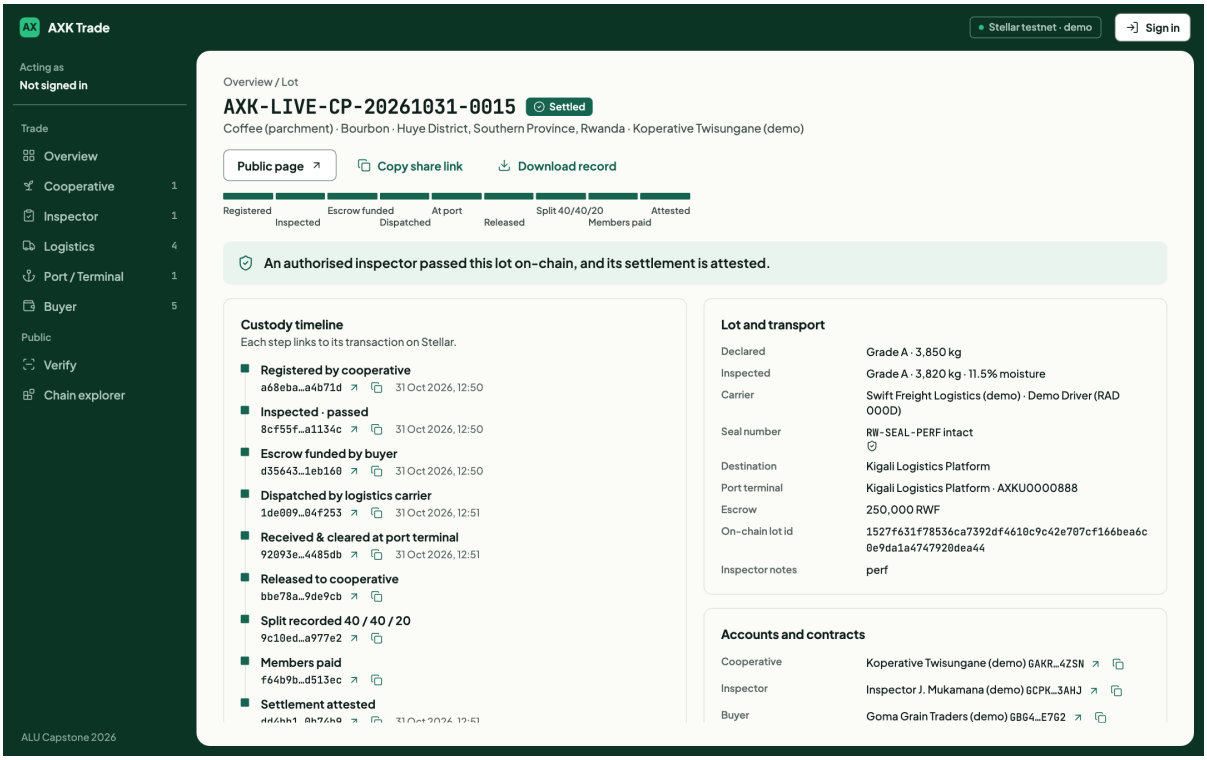


Figure 15: Lot page with the nine-step custody timeline of a settled trade

Public verification (FR8, FR10). Anyone can look up a lot by reference without an account (Figure 16). The public lot page (Figure 17) shows the verdict, the chain of custody with transaction links, the digests of each record, a QR code that opens the same page, and a button to download the lot's history as a GS1 EPCIS 2.0 document. It shows no names of drivers, inspectors or buyers, but it does show the carrier, vehicle plate and customs details.

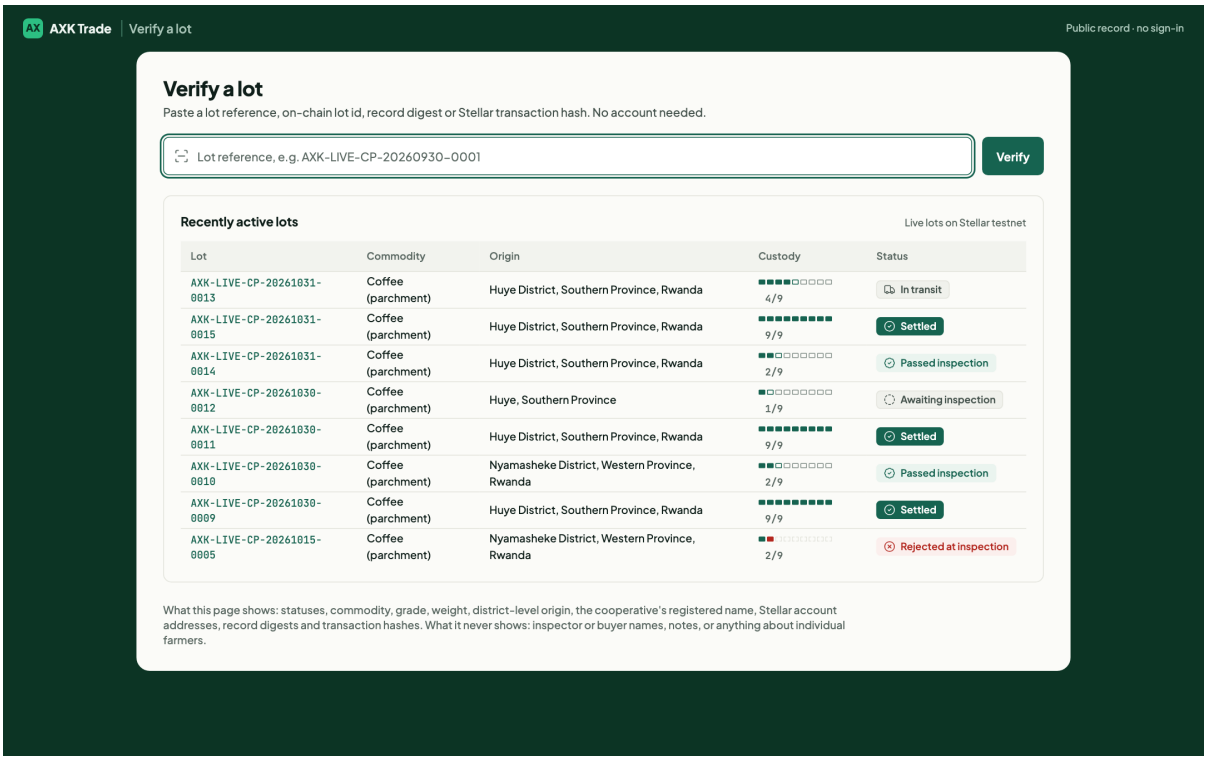


Figure 16: Public verification page

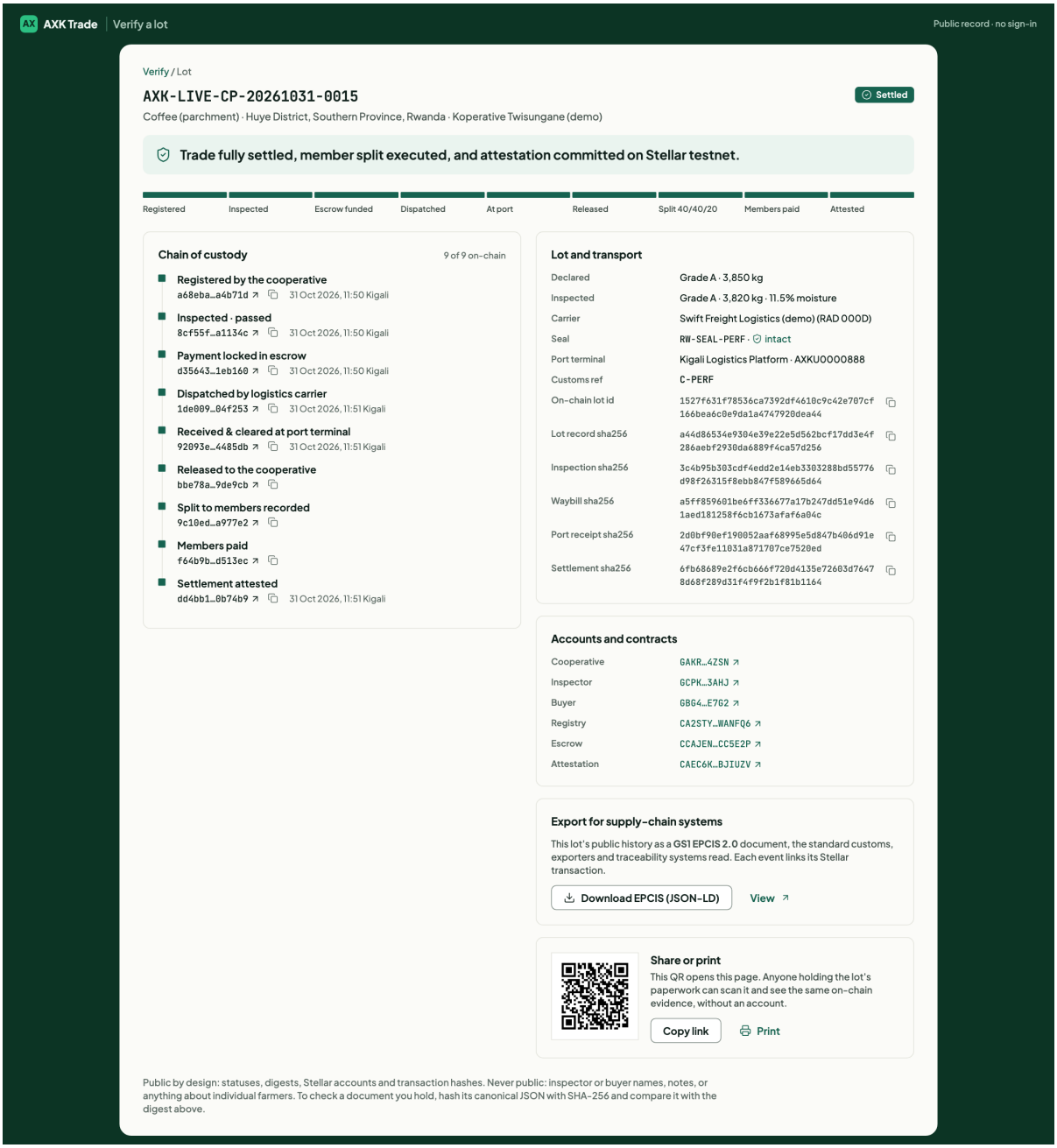


Figure 17: Public page for a settled lot with digests, QR code and EPCIS export

Chain explorer (FR11). Figure 18 shows the chain explorer, which reads events directly from the Stellar testnet: the latest ledger, escrows funded and released, the flow of funds from buyer to escrow to cooperative and members, and the list of contract events.

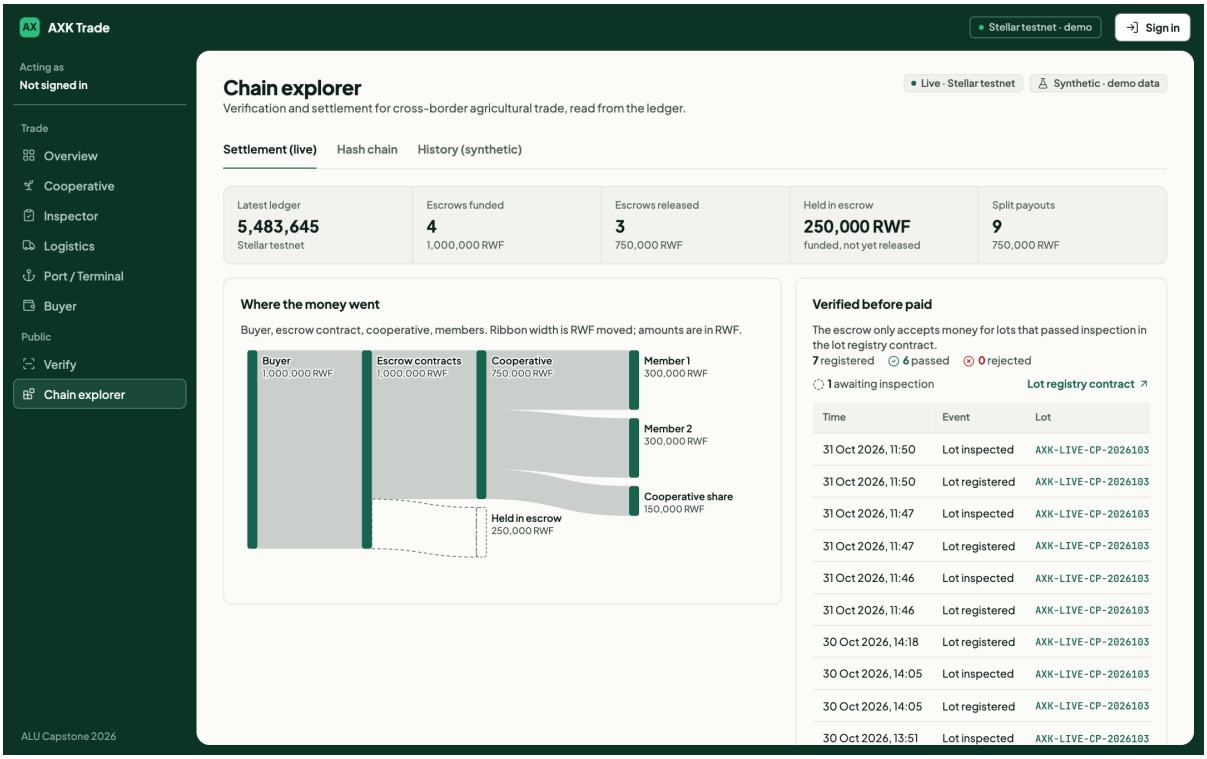


Figure 18: Chain explorer reading live contract events from the Stellar testnet

Phone layout. Figure 19 shows the cooperative screen and a public lot page at phone width (390 pixels). The layout adapts to one column and no content runs off the side of the screen.

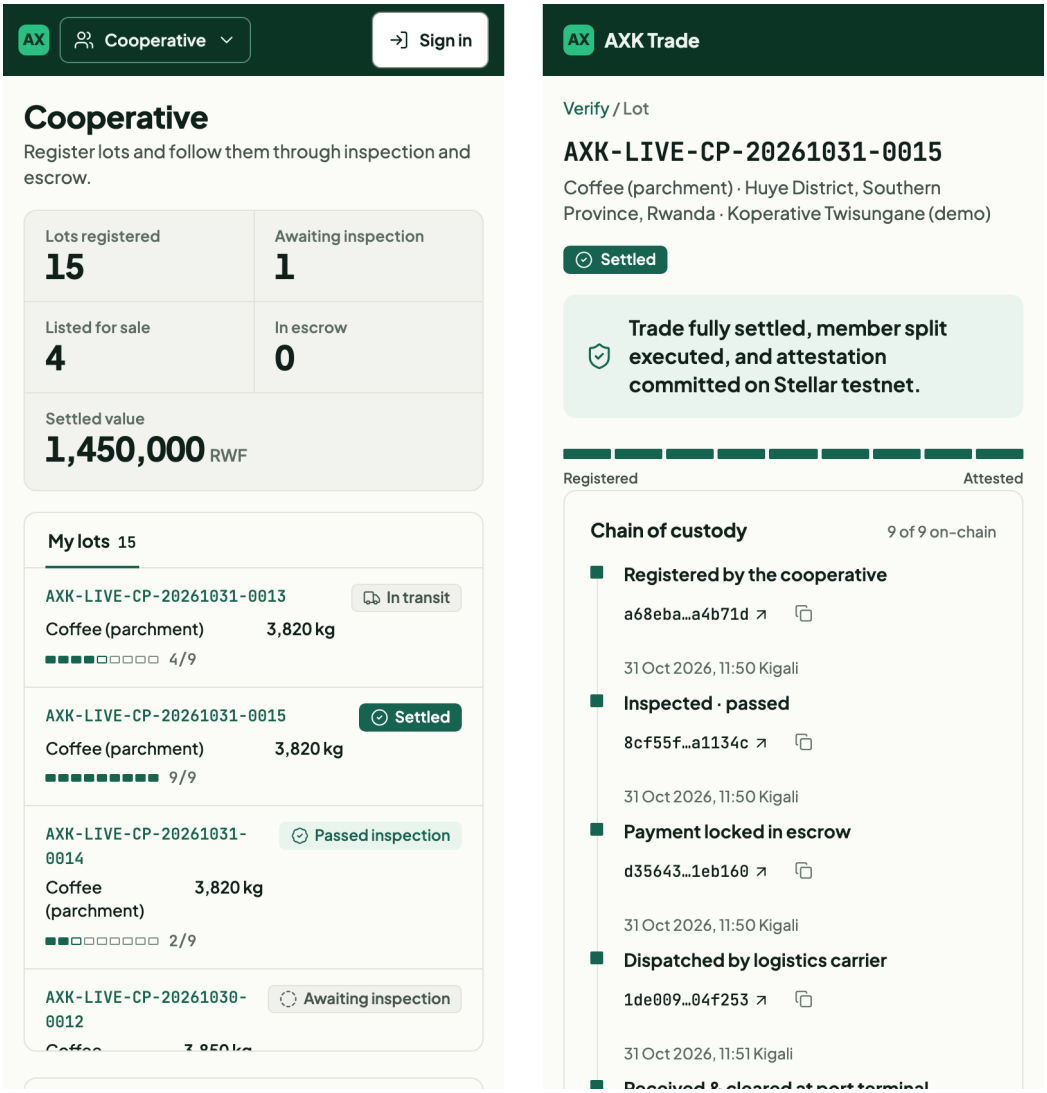


Figure 19: Cooperative screen and public lot page on a phone

4.3 Results

Fifteen lots were taken through the system on the Stellar testnet between 30 September and 31 October 2026 (Table 11). Six were settled end to end, three were rejected at inspection with a recorded reason, five passed inspection and were awaiting a buyer, and one was awaiting inspection at the time of writing. A further lot was placed in transit to demonstrate the seal check. Every on-chain step of the settled lots has a transaction that can be viewed on a public explorer, and for every settled lot all four records (lot, waybill, port receipt and settlement) matched their on-chain digests.

Table 11: Outcome of all lots run on the Stellar testnet

Outcome	Lots	Notes
Settled (paid, split and attested)	6	0001, 0004, 0008, 0009, 0011, 0015; all records matched the chain
Rejected at inspection	3	Moisture above tolerance (2), weight below declared (1)
Passed, awaiting a buyer	5	Listed for sale on the buyer screen
In transit	1	Used for the seal-mismatch test in Section 4.5.3
Awaiting inspection	1	In the inspector's queue
Total	**16**	Test RWF escrowed and released on settled lots: 1,450,000

Figure 20 shows the time each settled lot took on the testnet. From registration to settlement the six trades took between 74 and 236 seconds, with a median of 92 seconds; from funding the escrow to settlement they took 50 to 78 seconds. The first trade, on 30 September, took longest because the steps were carried out by hand with pauses between them; later trades were run by the measurement script. These times include the network's own confirmation time of about five seconds per ledger.

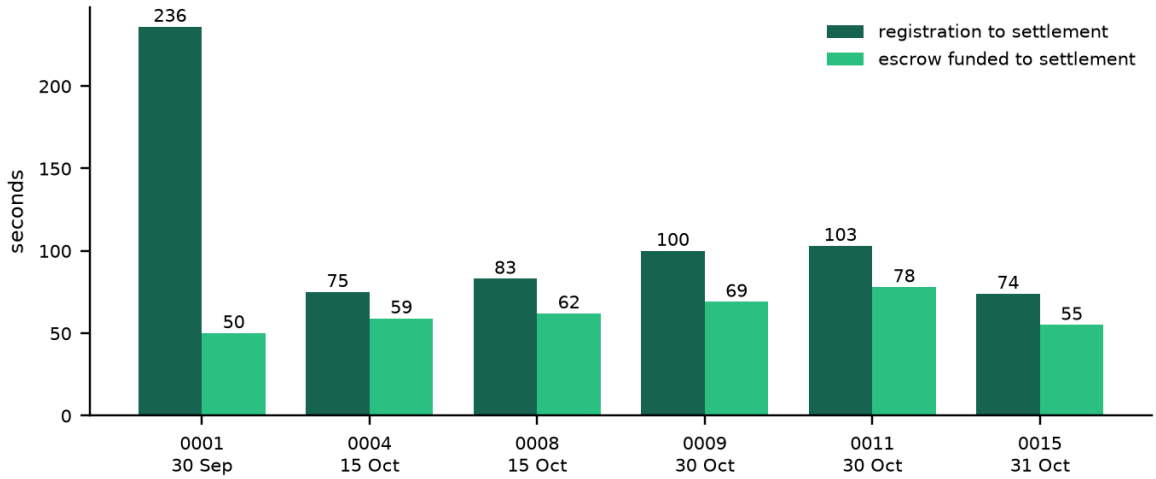


Figure 20: Time from registration and from escrow funding to settlement for each settled lot (seconds)

Figure 21 breaks one timed trade (lot 0015, 31 October) into steps. Settlement was the longest step, at 35.4 seconds, because it is four transactions: releasing the escrow, configuring the split, paying the members and attesting the settlement record. The full trade took 87.5 seconds, well inside the target of three minutes.

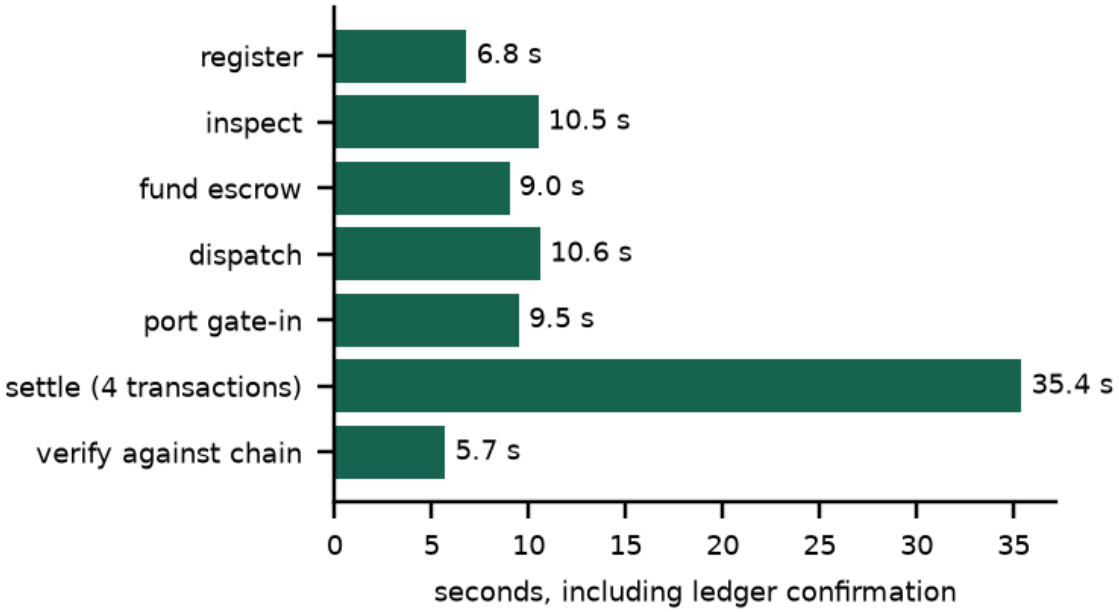


Figure 21: Seconds per step of one full trade on the Stellar testnet (lot 0015)

Figure 22 summarises the automated tests. All 169 tests passed on 31 October 2026: 65 contract tests, 22 tests of the ledger package, 58 tests of the custody display and demo logins, 7 end-to-end tests, 8 EPCIS tests and 9 ODK tests (one of which runs only against a live network and was skipped by design).

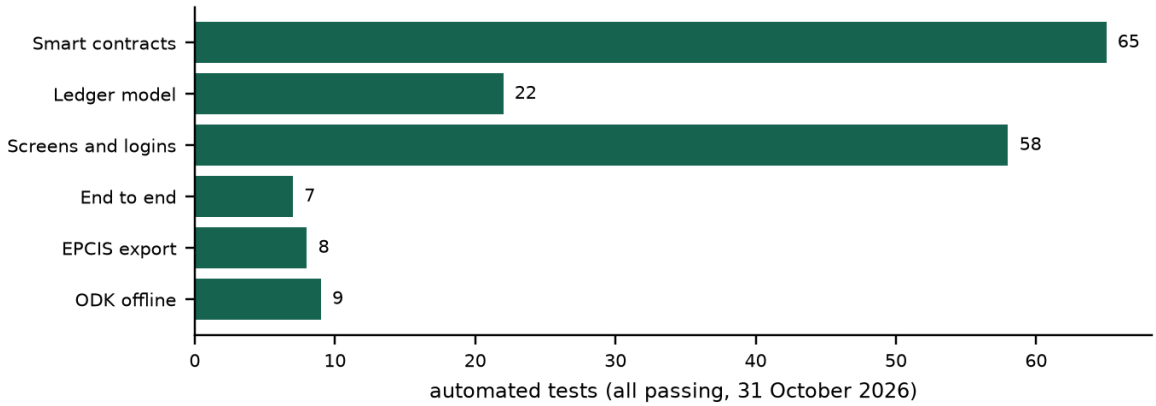


Figure 22: Automated tests per suite, all passing

4.4 Discussion of Findings

The first research question asked what verification and settlement failures traders face. The literature pointed to unverifiable quality, paper-based custody and slow, exclusionary payment (Affognon et al., 2015; World Bank, 2020). Participants confirmed this picture. The most common causes of delay in their last trade were quality disputes on arrival (15 of 20) and being unable to confirm the lot before paying (14), followed by currency exchange (12), waiting for payment to clear (11), paperwork and certificates (10) and transport or border queues (9). Twelve were paid by bank transfer, seven by mobile money and one in cash on

delivery, and the median time from agreeing the deal to receiving the money or goods was 30 days (range 10 to 90).

The second research question asked how a cryptographic layer could represent quality, quantity and provenance so that distant counterparties can trust it. The results showed a working answer: each record was made canonical and hashed, only the digest was placed on-chain, and every settled lot's records matched the chain. This kept trade data private while making any change detectable, which addresses the concern raised by Kamilaris et al. (2019) about sharing commercial data on a ledger. It also addressed the warning of Creydt and Fischer (2019) that a ledger cannot make a false input true: the system did not let the seller vouch for the goods; it required an independent, allow-listed inspector, and the contracts refused an inspection by the lot's own cooperative.

The third research question asked whether combining verification and settlement reduces completion time and increases trust. On the system side, a complete trade from registration to settlement took between 74 and 236 seconds on the testnet, against the days or weeks typical of current practice. Payment could not be released for an uninspected lot, could not be redirected, and could be refunded after the deadline, which removes the first-mover risk described in the baseline. This is the reduction in verification and enforcement cost that transaction cost economics predicts should widen trade beyond existing networks (Williamson, 1985). Participants reported a median of 30 days from agreeing a deal to receiving payment and rated trust in the workflow at 6.55 out of 7, although trust was measured after using demo accounts, not in live trade.

The findings also showed limits. Settlement speed on the testnet does not include the human time of inspection and transport, which dominate a real trade. The system held signing keys on behalf of each role, which is convenient for testing but means users must trust the operator, the same concern Saberi et al. (2019) and Kouhizadeh et al. (2021) raised about governance. And as Abebe and Gebresenbet (2022) warned, the gains depend on connectivity and digital literacy, which is why the offline inspection import and the low-end phone tests mattered. These limits are taken up in Chapter Five.

4.5 Testing

4.5.1 Introduction

This section reported the testing strategies, environments and results. Testing combined unit tests of the contracts and the ledger package, validation against the requirements, integration and end-to-end tests of the application, cross-browser and performance tests, and acceptance testing with participants. Table 12 lists the environments used.

Table 12: Testing environments

Item	Configuration
Development and test machine	macOS 15.5, Apple silicon (10 CPU cores), Homebrew
Hosted server	Ubuntu 24.04 virtual machine, nginx, supervisor, Frappe v15, MariaDB 10.11, Redis; capstone.axk.org with TLS
Runtime	Python 3.11, Rust 1.98, Stellar CLI 28.0.0, Node.js 24 (asset build)
Network	Stellar testnet (Soroban RPC and Horizon); RWF test token as a Stellar asset contract
Browsers	Chromium 156 (Chrome, Edge), Firefox 157, WebKit 27.2 (Safari), through Playwright
Device sizes	Phone 375 x 812, tablet 768 x 1024, desktop 1440 x 900; low-end profile with CPU slowed four times and a slow 3G connection

Error tracking. Errors were handled at three levels. The contracts returned typed errors, which the ledger package translated into plain messages using a separate error table for each contract. When a chain call failed, the application rolled back the database change, saved the reason in the lot's `last_error` field and showed it on screen, so a lot never claimed a step the ledger had not accepted. Server exceptions were written to Frappe's error log and to the web server log on the hosted machine, which the handover document tells the operator how to check.

4.5.2 Objective of Testing

The objective of testing was to show, with evidence, that the system met its functional requirements, behaved reliably when given wrong or unexpected input, worked across browsers and devices with acceptable speed, and served the research objectives in Chapter One: a tamper-evident verification model (objective 2), a working product that combines it with settlement (objective 3), and an evaluation with users (objectives 4 and 5).

4.5.3 Unit Testing Outputs

Unit tests checked each module in isolation. The 65 contract tests ran in a simulated Soroban environment and covered the 14 security guarantees listed in the repository's security document, each pinned by at least one test (Figure 23). All passed.



Figure 23: Output of the 65 smart contract unit tests

The 22 tests of the ledger package ran with the Stellar command-line tool replaced by a fake runner, so they needed no network (Figure 24). They checked that digests are unchanged by the restructuring, that out-of-order steps are refused, that a refused transaction leaves the trade unchanged, that settlement resumes correctly after a failure part-way, and that a buyer short of funds is told so before anything is signed.

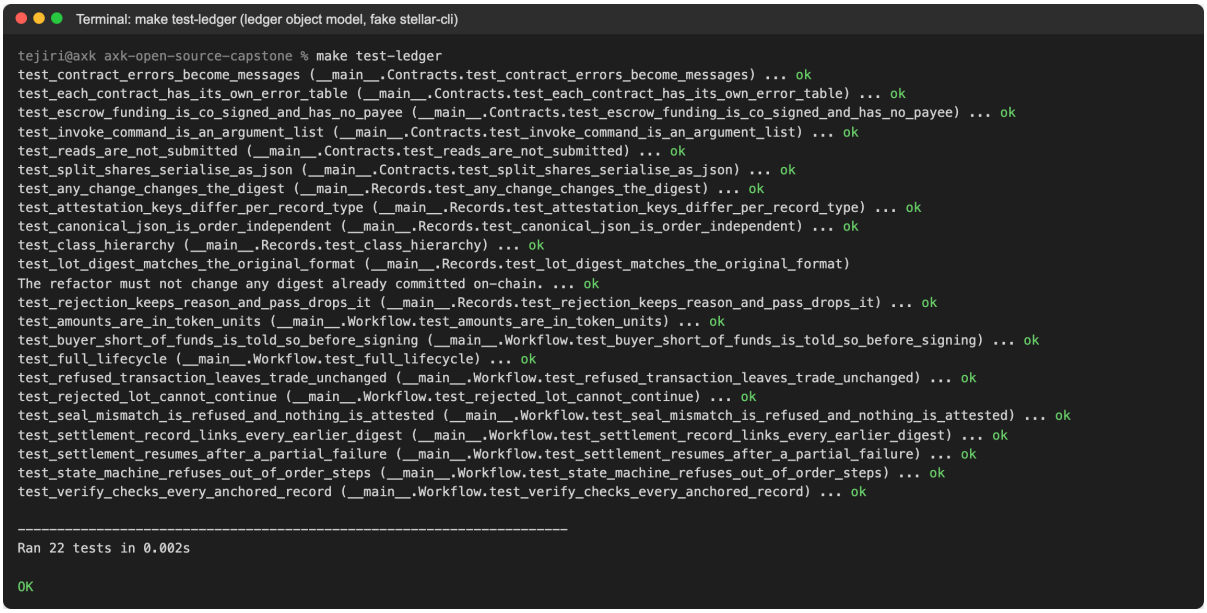


Figure 24: Output of the 22 ledger package unit tests

Error handling. The following screenshots show how the running system handled wrong input. In Figure 25, the port operator entered seal RW-SEAL-9999 for a lot dispatched under RW-SEAL-0013. The system raised a tamper alert, signed nothing and left the lot in transit.

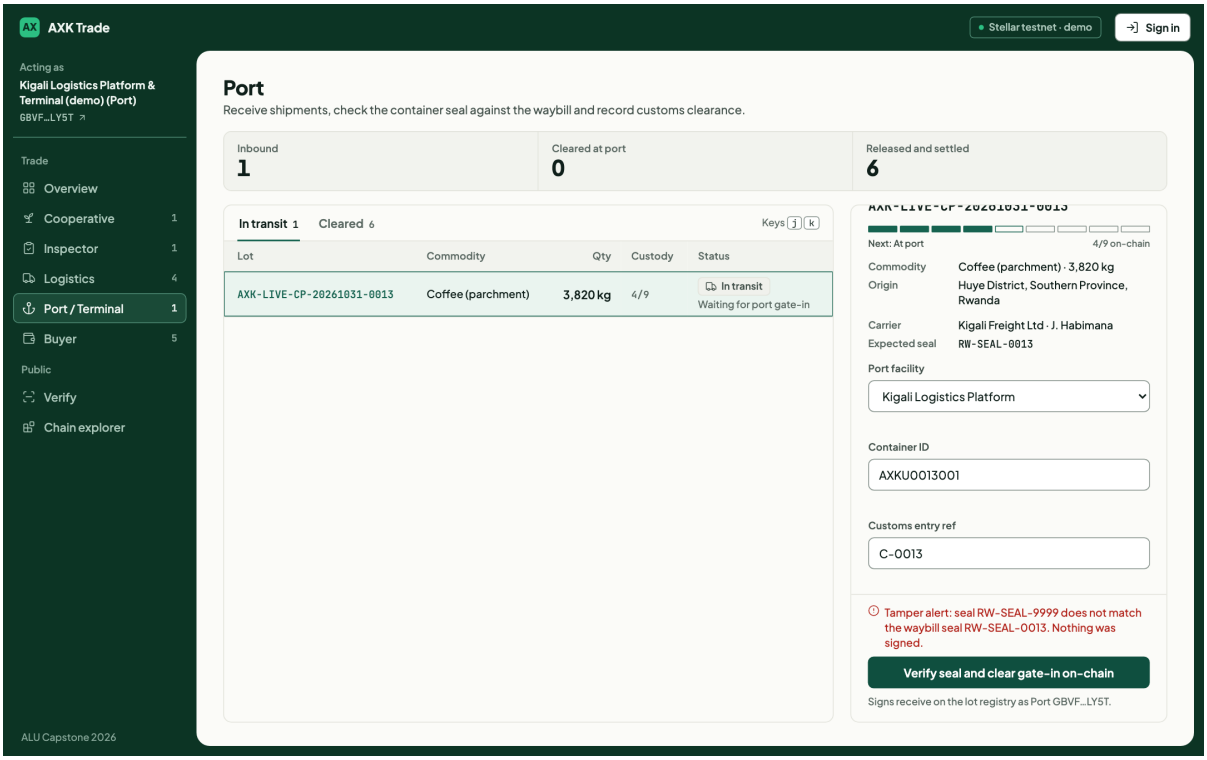


Figure 25: Tamper alert when the arriving seal does not match the waybill

In Figure 26, a buyer tried to fund 12,988,000 RWF with a wallet holding 1,700,001 RWF. The screen showed the balance and refused before contacting the network. Testing this case on the testnet uncovered a defect: the token contract's insufficient-balance error has the same number, 10, as the escrow's "lot has not passed inspection" error, so the earlier version

told the buyer the lot had failed inspection. The workflow was changed to check the balance first and a unit test was added (Listing 6).

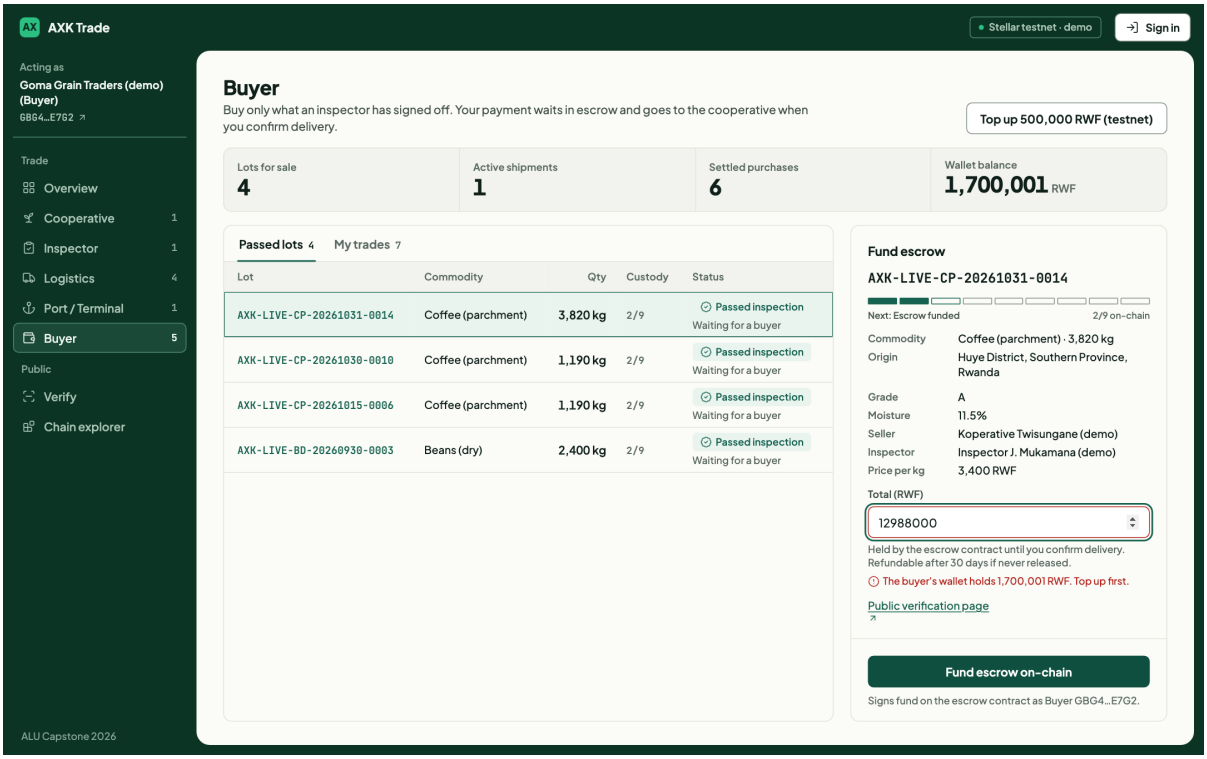


Figure 26: Funding refused because the buyer's balance is too low

Listing 6: Balance check added after testing (ledger/workflow.py)

```
if self.token:
    # The token contract's "insufficient balance" is also error #10, which would
otherwise be read
    # as the escrow's #10 ("lot has not passed inspection"), so check before signing.
    held = self.token.balance(self._sign("verifier"), trade.buyer_account)
    if held < self._units(amount_rwf):
        raise ChainError(f"The buyer holds {held / self.decimals:,.0f} RWF, less than
the "
                         f"{amount_rwf:,.0f} RWF to lock in escrow")
```

In Figure 27, the inspector tried to reject a lot without choosing a reason. The form refused and asked for a reason; the server enforces the same rule.

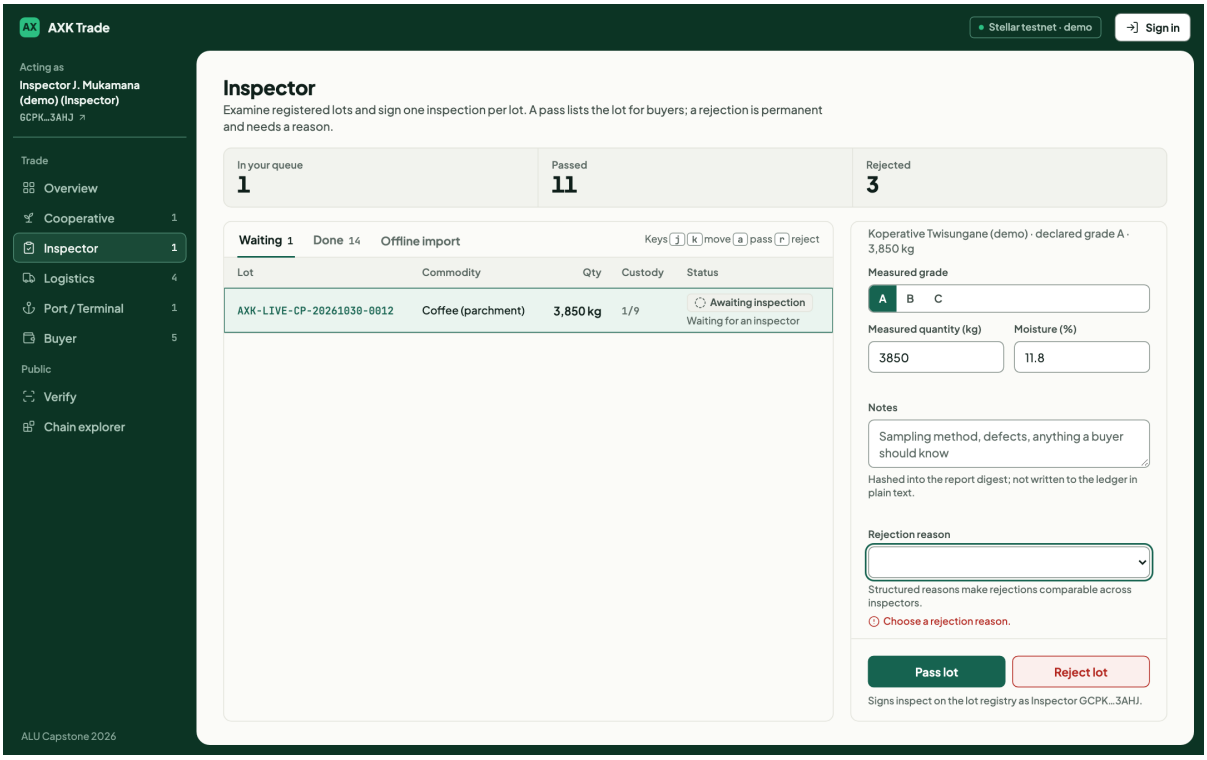


Figure 27: Rejection refused until a reason is chosen

The 58 tests of the custody display and demo logins (Figure 28) checked that a custody notch fills only for a valid 64-character transaction hash, that demo accounts are created only when a strong password is configured, that a demo user cannot change the shared password, account or sharing, that the sign-in page cannot be used to redirect to another site, and that each lot carries its nine transaction fields.

```
Terminal: make test-custody test-demo (Custody Strip, demo logins, tx fields)

tejiri@axk axk-open-source-capstone % make test-custody test-demo
..............
----------------------------------------------------------------------
Ran 15 tests in 0.000s

OK

test_cannot_change_or_disable_shared_account (__main__.Live.test_cannot_change_or_disable_shared_account) ... skipped 'set AXK_DEMO_LIVE=1 and AXK_DEMO_PASSWORD to run against a server'
test_cannot_escalate (__main__.Live.test_cannot_escalate) ... skipped 'set AXK_DEMO_LIVE=1 and AXK_DEMO_PASSWORD to run against a server'
test_cannot_rename_or_share_shared_account (__main__.Live.test_cannot_rename_or_share_shared_account) ... skipped 'set AXK_DEMO_LIVE=1 and AXK_DEMO_PASSWORD to run against a server'
test_demo_accounts_matches_flag (__main__.Live.test_demo_accounts_matches_flag) ... skipped 'set AXK_DEMO_LIVE=1 and AXK_DEMO_PASSWORD to run against a server'
test_each_login_lands_on_its_screen_and_stays_out_of_desk (__main__.Live.test_each_login_lands_on_its_screen_and_stays_out_of_desk) ... skipped 'set AXK_DEMO_LIVE=1 and AXK_DEMO_PASSWORD to run against a server'
Ran 40 tests in 0.041s
OK (skipped=5)
AXK demo users skipped: site config axk_demo_password is unset or under 8 characters
AXK demo users skipped: site config axk_demo_password is unset or under 8 characters
AXK demo users skipped: site config axk_demo_password is unset or under 8 characters
AXK demo users skipped: site config axk_demo_password is unset or under 8 characters
AXK demo users skipped: site config axk_demo_password is unset or under 8 characters
AXK demo users skipped: site config axk_demo_password is unset or under 8 characters

../Users/tejiriagent/Documents/axk-open-source-capstone/scripts/test_trade_txs.py:73: ResourceWarning: unclosed file <_io.TextIOWrapper name='/Users/tejiriagent/Documents/axk-open-source-capstone/frappe_app/axk_capstone/axk_capstone/verify.py' mode='r' encoding='utf-8'>
  src = open(os.path.join(ROOT, "frappe_app/axk_capstone/axk_capstone/verify.py")).read()
ResourceWarning: Enable tracemalloc to get the object allocation traceback
.
----------------------------------------------------------------------
Ran 3 tests in 0.004s

OK
```

Figure 28: Output of the custody display, demo login and transaction field tests

4.5.4 Validation Testing Outputs

Validation testing checked the system against the requirements and research objectives. Table 13 maps each requirement to the test that validated it and compares the expected and actual results.

Table 13: Requirements traceability and validation results

Requirement	Test case	Expected	Actual
FR1 Role access	Guest calls a signing endpoint; demo user tries to change password	Refused	Refused (demo user tests)
FR2 Register lot	Register lot; register same reference again	Lot on-chain; duplicate refused (error 1)	As expected (contract and end-to-end tests)
FR3 Inspect	Pass with values; reject with no reason; cooperative inspects own lot	Pass recorded; others refused	As expected; own-lot inspection refused with error 3
FR4 Fund escrow	Fund an uninspected lot; fund a passed lot; fund beyond balance	Refused; funded; refused with balance message	As expected (testnet error 10; Figure 26)
FR5 Dispatch	Dispatch a funded lot; dispatch an unfunded lot	Attested; refused	As expected
FR6 Gate-in	Matching seal; mismatched seal	Attested; tamper alert, nothing signed	As expected (Figure 25)
FR7 Settle	Confirm delivery on a lot at port	Release, split, pay, attest; parts sum to amount	Six lots settled; split lossless
FR8 Public trace	Open lot page without login; scan QR	Lot shown with no personal data	As expected
FR9 Verify	Re-check records of settled lots	All records match	All four records matched for all six
FR10 EPCIS	Export a lot and validate against the GS1 schema	Valid EPCIS 2.0 document	8 of 8 tests passed
FR11 Explorer	Open chain explorer	Live events from testnet	As expected (Figure 18)

4.5.5 Integration Testing Outputs

Integration testing checked the interaction between the screens, the API, the database, the ledger package and the contracts. The seven end-to-end tests exercised every API endpoint over HTTP: the overview and personas, a full lifecycle from registration to settlement, inspection rejection, seal tampering, invalid state transitions, a lot that does not exist, and the web routes and trace page. The EPCIS tests validated the exported document against the official GS1 schema, and the ODK tests covered the offline form, the importer and synchronisation with ODK Central, importing each submission once and skipping rejected ones (Figure 29).

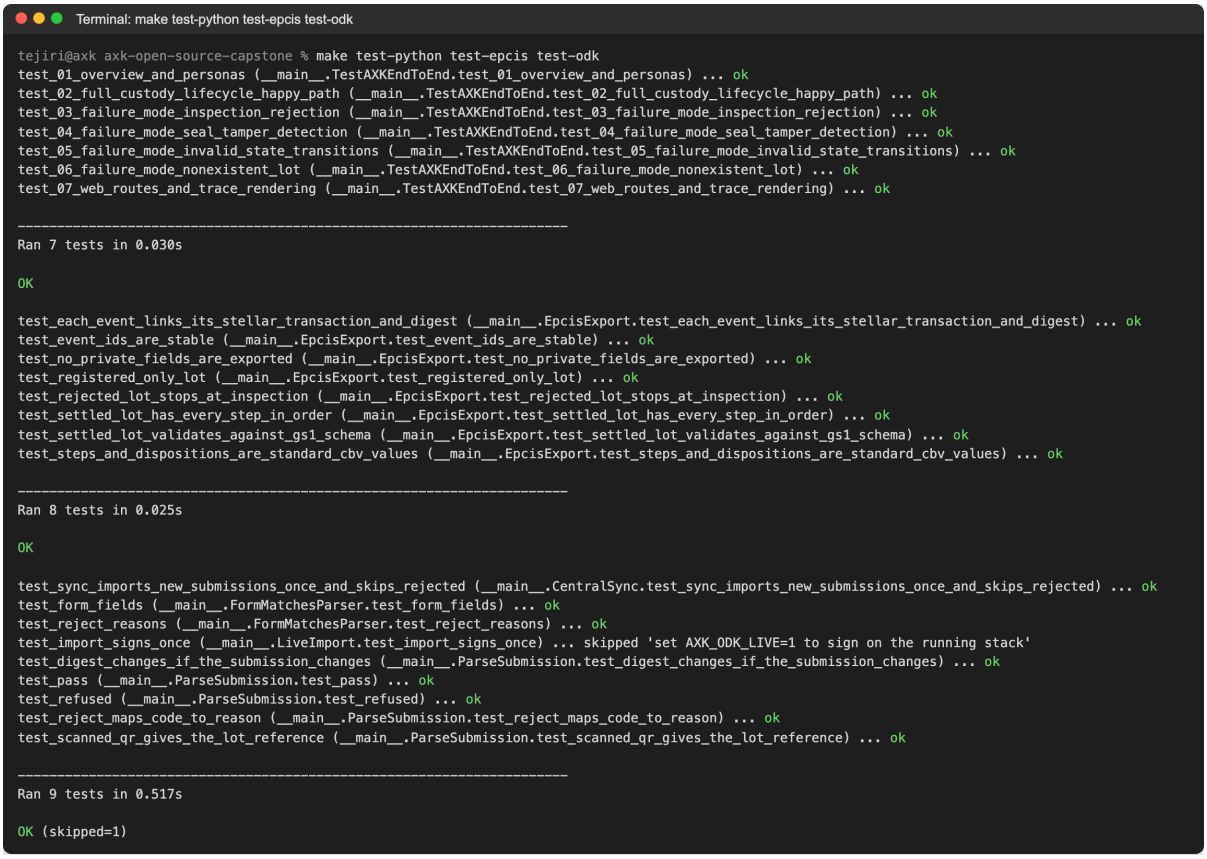


Figure 29: Output of the end-to-end, EPCIS and ODK tests

Data flow integrity was confirmed on the testnet itself. For each settled lot, the system re-hashed the lot record, waybill, port receipt and settlement record stored in MariaDB and asked the registry and attestation contracts whether the digests matched. All matched. The cross-contract call from the escrow to the registry was confirmed both ways: funding an approved lot succeeded, and funding an uninspected lot was refused by the contract with error 10, which is recorded with its transaction link in the repository's deployment notes.

4.5.6 Functional and System Testing Results

System testing measured performance and compatibility. Table 14 lists the performance results, measured on the local full stack with a warm cache over 20 requests each, and one full trade on the testnet.

Table 14: Performance and resource use

Measure	Result
Role screen (/trade), median	38.6 ms (95th percentile 39.6 ms)
Public lot page with QR code, median	50.7 ms (95th percentile 53.0 ms)
API calls (overview, lot, personas, EPCIS export), median	3.3 to 3.8 ms
One full trade on testnet (registration to verification)	87.5 s, of which settlement (four transactions) 35.4 s
Memory of the whole stack at rest	About 263 MB (web, workers, Redis, socket server, MariaDB)
Main script size (compressed)	20.1 KB; no frontend framework
Largest contract (WebAssembly)	7.4 KB

Compatibility was tested by loading all eight screens in three browser engines at three sizes and in a low-end profile, 80 page loads in total (Table 15 and Figure 30). All 80 loaded and rendered, with no JavaScript errors and no content running off the side of the screen. Median ready times were 61 to 111 milliseconds on a normal connection. On the low-end phone profile the median was 720 milliseconds and the slowest screen took 3.8 seconds, inside the five-second target. Before a lean page template was introduced, the same profile took 57.7 seconds for the first page because every screen loaded 3.9 MB of unused framework files; finding and fixing this was a direct result of the test.

Table 15: Compatibility results (31 October 2026)

Engine	Size	Profile	Pages OK	Median ready	Slowest	JS errors	Overflow
Chromium	Phone	Normal	8/8	70 ms	140 ms	0	0
Chromium	Tablet	Normal	8/8	66 ms	80 ms	0	0
Chromium	Desktop	Normal	8/8	61 ms	76 ms	0	0
Firefox	Phone	Normal	8/8	111 ms	187 ms	0	0
Firefox	Tablet	Normal	8/8	111 ms	176 ms	0	0
Firefox	Desktop	Normal	8/8	110 ms	168 ms	0	0
WebKit	Phone	Normal	8/8	73 ms	142 ms	0	0
WebKit	Tablet	Normal	8/8	74 ms	89 ms	0	0
WebKit	Desktop	Normal	8/8	72 ms	86 ms	0	0
Chromium	Phone	4x CPU, slow 3G	8/8	720 ms	3,786 ms	0	0

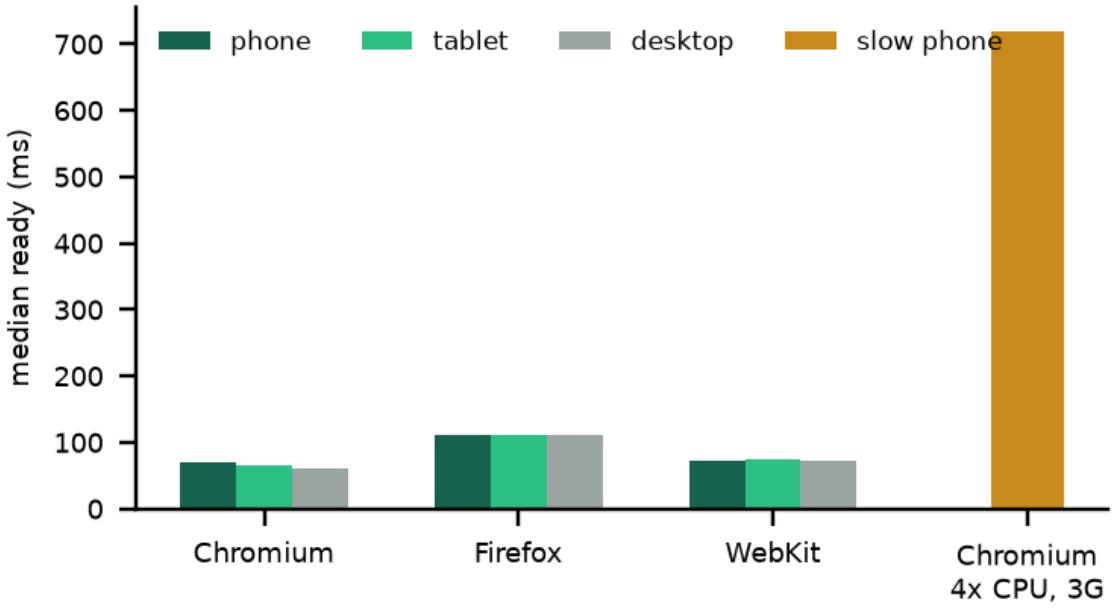


Figure 30: Median screen ready time by browser engine and screen size

4.5.7 Acceptance Testing Report

Acceptance testing evaluated whether prospective users found the system useful, easy to use and trustworthy, and whether they intended to use it. Participants used, or in one case watched, the role screen matching their work on the hosted system and then completed the questionnaire described in Section 3.2. The raw survey data and the signed consent records are kept by the researcher and summarised in Appendix C.

Table 16 profiles the participants.

Table 16: Participant profile

Characteristic	Category	Number	Percent
Role used	Cooperative (registered a lot)	8	40
	Buyer (paid into escrow, confirmed delivery)	7	35
	Logistics carrier (dispatched a shipment)	4	20
	Observer (watched the full demonstration)	1	5
	Inspector or port operator	0	0
Description	Cooperative officer	7	35
	Trader or buyer	6	30
	Transport or logistics	5	25
	Cooperative member	1	5
	Student with trade or agribusiness experience	1	5
Years in produce trade	Less than 1	7	35
	1 to 3	1	5
	4 to 10	3	15
	More than 10	9	45
Traded across a border	Yes	16	80
	No	4	20
Uses mobile money	Yes, regularly	15	75
	Sometimes	5	25
Main device	Smartphone	11	55
	Computer	5	25
	Basic or feature phone	4	20
Total participants		**20**	100

Table 17 reports the construct scores on the seven-point scale, where 4 is neutral, with Cronbach's alpha for each construct.

Table 17: Technology acceptance results

Construct	Items	Mean (out of 7)	Standard deviation	Cronbach's alpha
Perceived usefulness	PU1 to PU4	6.30	0.71	0.81
Perceived ease of use	EOU1 to EOU4	6.35	0.72	0.90
Trust	T1 to T4	6.55	0.62	0.92
Behavioural intention	BI1 to BI3	6.83	0.32	0.59

All four constructs were well above the neutral midpoint of 4, and no participant's average on any construct was below 5. Behavioural intention scored highest (6.83) and perceived usefulness lowest (6.30); the lowest single answer, a 3, was for PU3, whether the system would reduce the risk of not being paid or not receiving what was paid for. Cronbach's alpha exceeded 0.7 for usefulness, ease of use and trust, but not for behavioural intention (0.59), so the intention score should be read with caution. Answers were concentrated at the top of the scale: 63 percent of all item answers were 7, only two were 4 or lower, and three participants gave the same answer to every item. Participants reported that their last trade took a median of 30 days (range 10 to 90) from agreeing the deal to receiving the money or goods, with the main causes of delay being quality disputes on arrival, being unable to confirm the lot before paying, currency exchange and waiting for payment to clear. In the AXK workflow the equivalent on-chain steps took between 74 and 236 seconds.

Only three participants answered the open questions on usefulness, barriers and changes, so these themes are indicative only. All three named tracking or traceability as the most useful feature, for example "traceability and the faster payment system" (AXK1). The barrier named was buyers not adopting the system (AXK1, AXK6), and the one requested change was more training time (AXK1); the other two wrote that nothing needed to change. Comments on participants' last trade described goods going missing or stolen in transit (AXK4, AXK6, AXK8), recurring quality problems (AXK10), payment arriving in parts (AXK1), difficulty obtaining trade finance (AXK5) and the burden of complying with the EU Deforestation Regulation when sourcing from Africa (AXK11).

The results suggest that prospective users found the workflow useful, easy to use and trustworthy, and that the problems they reported, disputed quality, unverified lots and slow payment, are the ones the inspection-gated escrow and the public record are designed to address. They are early signals rather than proof: the sample of 20 was below the 30 to 50 planned in the proposal, testing took place on one day with demo accounts, no inspector or port operator took part, and the high, tightly clustered scores may partly reflect a positive response to a demonstration. A field pilot with real trades is needed to test the effect.

CHAPTER FIVE: SUMMARY, CONCLUSION AND RECOMMENDATIONS

5.1 Summary

This study set out to address a specific gap in East African agricultural trade: no system available to smallholder cooperatives and small traders verified a commodity and settled its payment in one trusted workflow. Following a design science approach and an iterative development model, it built AXK Network, an open-source layer made of four Soroban smart contracts on the Stellar testnet and a Frappe web application with a screen for each trade role, a public verification page and a chain explorer. Each custody step, from registration through inspection, escrow, dispatch and gate-in to settlement, became a signed transaction, and only digests of the underlying records were placed on-chain.

The system was evaluated technically and with prospective users. All 169 automated tests passed; all 80 page loads across three browser engines, three screen sizes and a low-end phone profile rendered without errors; and fifteen lots were run on the testnet, six of which settled in 74 to 236 seconds from registration with every record matching the chain. Testing led to concrete improvements, including a 57.7-second first page load on slow phones reduced to under four seconds and a misleading payment error corrected. The system was deployed at capstone.axk.org with one demo account per role. Table 18 summarises how far each objective was met.

Table 18: Achievement of the specific objectives

Objective	Status	Evidence
1. Assess verification and settlement failures	Met through literature and participant reports	Chapter Two; baseline questions in acceptance testing (Section 4.5.7)
2. Design a tamper-evident verification model	Met	Canonical records, digests on-chain, inspection-gated escrow; all settled records matched the chain
3. Develop an MVP combining verification and settlement	Met on testnet	Four contracts, Frappe app, hosted system; mobile-money payout designed but not built
4. Test the MVP with traders and cooperative members	Partly met	20 participants in acceptance testing (cooperatives, buyers and carriers; no inspectors or port operators); the three-month field pilot is still to run
5. Evaluate and recommend a scaling path	Met	Chapter Four results; recommendations in Section 5.3

5.2 Conclusion

The results showed that verification and settlement can be combined in a single workflow that both trading parties, and outsiders, can check. Making an independent inspection the condition for payment, and letting the escrow contract rather than the buyer decide who is paid, removed the first-mover risk that forces cooperatives and buyers to trust each other blindly. Recording each custody step as a digest kept commercial data private while making any later change detectable. On the testnet, a complete trade took less than four

minutes of processing, compared with the days or weeks of current practice, at no real transaction cost.

Acceptance testing with 20 prospective users gave high mean scores out of 7 for usefulness (6.30), ease of use (6.35), trust (6.55) and intention to use (6.83), in line with the Technology Acceptance Model's expectation that a system seen as useful and easy to use will be adopted, although the small sample and tightly clustered scores make this an early signal rather than a test of the model. The study therefore supported the expectation, drawn from transaction cost and institutional theory, that a shared, rule-enforcing ledger can lower the cost of verifying and enforcing trade between parties who do not know each other. It did not yet show the effect in live trade with real money, which requires the field pilot and the changes described below.

5.3 Recommendations

For cooperatives and traders. Cooperatives can use the public lot page and QR code as a portable proof of inspection when negotiating with buyers they have not traded with before, and buyers can make inspection-gated escrow a condition of purchase, which protects both sides without a bank guarantee.

For AXK Network and other implementers. Before handling real money, the system should move from custodial keys to signing by each participant, with the carrier and port signing their own attestations; the contracts should be audited externally; the seal check should be enforced in a contract rather than only in the application; and payout to mobile money should be connected through a regulated anchor. An event indexer should be added so that the explorer's history does not depend on the network's short event retention.

For policymakers and AfCFTA implementers. Payment infrastructure such as PAPSS would gain from a standard way to attach verified commodity records to a payment. Supporting an open, standards-based format such as GS1 EPCIS for inspection and custody records, and recognising independent inspection records in trade finance and customs, would let systems like this one interoperate rather than compete.

5.4 Limitations

- The system ran on the Stellar testnet with a test token; no real money moved, so behaviour under real financial risk was not observed.
- Signing keys were held by the server for each role. This was suitable for testing but requires users to trust the operator.
- The carrier's and port's attestations were signed by the verifier on their behalf, and the seal match was enforced in the application rather than in a contract.
- Acceptance testing used demo accounts on a single day, 8 November 2026, not repeated trades over three months. The sample was 20 participants, below the 30 to 50 planned; no inspector or port operator took part; and scores were close to the top of the scale, which limits generalisation.
- The live explorer reads events from the network's RPC service, which keeps about seven days of history; older history in the explorer is clearly labelled synthetic data.

- The interface was in English only, which excludes some users in the target corridor.

5.5 Contributions to the Body of Knowledge

The study contributed a working, open-source design in which commodity verification gates settlement on a public ledger, a combination the reviewed literature treated as separate problems. It showed a concrete way to place trade records on a ledger without exposing commercial data, by committing canonical digests and letting any holder of a record re-check it. It added East African evidence, from testnet trades and from prospective users, to a literature that is mostly global or conceptual. And it documented practical findings that other builders can reuse, such as the need to check balances before an escrow call because error codes from nested contracts can be misread, and the large gains from removing unused framework code for users on slow phones.

5.6 Suggestions for Further Studies

- Run the planned three-month pilot with 30 to 50 traders and at least one cooperative on a single corridor, measuring completion time, disputes and trust against matched trades outside the system.
- Compare custodial and non-custodial signing for smallholder users, including recovery of lost keys and the use of basic phones.
- Connect settlement to mobile-money wallets through a regulated anchor or a Mojaloop-based switch, and study the cost and time of cashing out.
- Study the governance of the inspector allow-list: who appoints inspectors, how they are paid and how their accuracy is audited.
- Translate the screens into Kinyarwanda, French and Swahili and test whether this changes ease of use and adoption.
- Extend the model to other commodities and corridors, and to trade finance, where verified inventory could support credit to cooperatives.

REFERENCES

Abebe, M. S., & Gebresenbet, G. (2022). The role of digital agriculture in enhancing food systems in East Africa. *Food Security, 14*, 939-953. https://doi.org/10.1007/s12571-022-01271-2

Affognon, H., Mutungi, C., Sanginga, P., & Borgemeister, C. (2015). Unpacking postharvest losses in sub-Saharan Africa: A meta-analysis. *World Development, 66*, 49-68. https://doi.org/10.1016/j.worlddev.2014.08.002

African Export-Import Bank. (2024). *African trade report 2024*. African Export-Import Bank.

African Union. (2018). *Agreement establishing the African Continental Free Trade Area*. African Union.

A.P. Moller - Maersk. (2022, November 29). *A.P. Moller - Maersk and IBM to discontinue TradeLens, a blockchain-enabled global trade platform* [Press release]. https://www.maersk.com/news/articles/2022/11/29/maersk-and-ibm-to-discontinue-tradelens

Brooke, J. (1996). SUS: A "quick and dirty" usability scale. In P. W. Jordan, B. Thomas, B. A. Weerdmeester, & I. L. McClelland (Eds.), *Usability evaluation in industry* (pp. 189-194). Taylor & Francis.

Buterin, V. (2014). *Ethereum: A next-generation smart contract and decentralized application platform* [White paper]. https://ethereum.org/en/whitepaper/

Casino, F., Dasaklis, T. K., & Patsakis, C. (2019). A systematic literature review of blockchain-based applications: Current status, classification and open issues. *Telematics and Informatics, 36*, 55-81. https://doi.org/10.1016/j.tele.2018.11.006

Coase, R. H. (1937). The nature of the firm. *Economica, 4*(16), 386-405. https://doi.org/10.1111/j.1468-0335.1937.tb00002.x

Cole, R., Stevenson, M., & Aitken, J. (2019). Blockchain technology: Implications for operations and supply chain management. *Supply Chain Management: An International Journal, 24*(4), 469-483. https://doi.org/10.1108/SCM-09-2018-0309

Creydt, M., & Fischer, M. (2019). Blockchain and more - Algorithm driven food traceability. *Food Control, 105*, 45-51. https://doi.org/10.1016/j.foodcont.2019.05.019

Cronbach, L. J. (1951). Coefficient alpha and the internal structure of tests. *Psychometrika, 16*(3), 297-334. https://doi.org/10.1007/BF02310555

Davis, F. D. (1989). Perceived usefulness, perceived ease of use, and user acceptance of information technology. *MIS Quarterly, 13*(3), 319-340. https://doi.org/10.2307/249008

Fofack, H., Dzene, R., & Hussein, O. A. M. (2021). Estimating the effect of AfCFTA on intra-African trade using augmented GE-PPML. *Journal of African Trade, 8*(2 Suppl.), 62-76. https://doi.org/10.2991/jat.k.211122.001

Food and Agriculture Organization of the United Nations. (2019). *The state of food and agriculture 2019: Moving forward on food loss and waste reduction*. Food and Agriculture Organization of the United Nations.

Galvez, J. F., Mejuto, J. C., & Simal-Gandara, J. (2018). Future challenges on the use of blockchain for food traceability analysis. *TrAC Trends in Analytical Chemistry, 107*, 222-232. https://doi.org/10.1016/j.trac.2018.08.011

Gefen, D., Karahanna, E., & Straub, D. W. (2003). Trust and TAM in online shopping: An integrated model. *MIS Quarterly, 27*(1), 51-90. https://doi.org/10.2307/30036519

Hartung, C., Lerer, A., Anokwa, Y., Tseng, C., Brunette, W., & Borriello, G. (2010). Open Data Kit: Tools to build information services for developing regions. In *Proceedings of the 4th ACM/IEEE International Conference on Information and Communication Technologies and Development* (Article 18). ACM. https://doi.org/10.1145/2369220.2369236

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly, 28*(1), 75-105. https://doi.org/10.2307/25148625

International Chamber of Commerce. (2007). *ICC uniform customs and practice for documentary credits (UCP 600)*. ICC Publication No. 600.

Jack, W., & Suri, T. (2014). Risk sharing and transactions costs: Evidence from Kenya's mobile money revolution. *American Economic Review, 104*(1), 183-223. https://doi.org/10.1257/aer.104.1.183

Jensen, T., Hedman, J., & Henningsson, S. (2019). How TradeLens delivers business value with blockchain technology. *MIS Quarterly Executive, 18*(4), 221-243. https://doi.org/10.17705/2msqe.00018

Kamath, R. (2018). Food traceability on blockchain: Walmart's pork and mango pilots with IBM. *The Journal of the British Blockchain Association, 1*(1). https://doi.org/10.31585/jbba-1-1-(10)2018

Kamilaris, A., Fonts, A., & Prenafeta-Boldu, F. X. (2019). The rise of blockchain technology in agriculture and food supply chains. *Trends in Food Science & Technology, 91*, 640-652. https://doi.org/10.1016/j.tifs.2019.07.034

Kouhizadeh, M., Saberi, S., & Sarkis, J. (2021). Blockchain technology and the sustainable supply chain: Theoretically exploring adoption barriers. *International Journal of Production Economics, 231*, 107831. https://doi.org/10.1016/j.ijpe.2020.107831

Kshetri, N. (2018). Blockchain's roles in meeting key supply chain management objectives. *International Journal of Information Management, 39*, 80-89. https://doi.org/10.1016/j.ijinfomgt.2017.12.005

Larman, C., & Basili, V. R. (2003). Iterative and incremental development: A brief history. *Computer, 36*(6), 47-56. https://doi.org/10.1109/MC.2003.1204375

Lin, Q., Wang, H., Pei, X., & Wang, J. (2019). Food safety traceability system based on blockchain and EPCIS. *IEEE Access, 7*, 20698-20707. https://doi.org/10.1109/ACCESS.2019.2897792

Mavilia, R., & Pisani, R. (2022). Blockchain for agricultural sector: The case of South Africa. *African Journal of Science, Technology, Innovation and Development, 14*(3), 845-851. https://doi.org/10.1080/20421338.2021.1908660

Mazières, D. (2015). *The Stellar consensus protocol: A federated model for internet-level consensus*. Stellar Development Foundation. https://www.stellar.org/papers/stellar-consensus-protocol

Mojaloop Foundation. (2023). *Mojaloop documentation*. https://docs.mojaloop.io

Nakamoto, S. (2008). *Bitcoin: A peer-to-peer electronic cash system*. https://bitcoin.org/bitcoin.pdf

North, D. C. (1990). *Institutions, institutional change and economic performance*. Cambridge University Press.

Osabutey, E. L. C., & Jackson, T. (2024). Mobile money and financial inclusion in Africa: Emerging themes, challenges and policy implications. *Technological Forecasting and Social Change, 202*, 123339. https://doi.org/10.1016/j.techfore.2024.123339

Pan-African Payment and Settlement System. (2021). *How it works*. PAPSS. https://papss.com/how-it-works/

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems, 24*(3), 45-77. https://doi.org/10.2753/MIS0742-1222240302

Republic of Rwanda. (2021). *Law No. 058/2021 of 13/10/2021 relating to the protection of personal data and privacy*. Official Gazette of the Republic of Rwanda.

Saberi, S., Kouhizadeh, M., Sarkis, J., & Shen, L. (2019). Blockchain technology and its relationships to sustainable supply chain management. *International Journal of Production Research, 57*(7), 2117-2135. https://doi.org/10.1080/00207543.2018.1533261

Stellar Development Foundation. (2024). *Smart contracts on Stellar (Soroban): Developer documentation*. https://developers.stellar.org/docs/build/smart-contracts

Suri, T., & Jack, W. (2016). The long-run poverty and gender impacts of mobile money. *Science, 354*(6317), 1288-1292. https://doi.org/10.1126/science.aah5309

Szabo, N. (1997). Formalizing and securing relationships on public networks. *First Monday, 2*(9). https://doi.org/10.5210/fm.v2i9.548

Venkatesh, V., & Davis, F. D. (2000). A theoretical extension of the technology acceptance model: Four longitudinal field studies. *Management Science, 46*(2), 186-204. https://doi.org/10.1287/mnsc.46.2.186.11926

Williamson, O. E. (1985). *The economic institutions of capitalism*. Free Press.

World Bank. (2020). *The African Continental Free Trade Area: Economic and distributional effects*. World Bank. https://doi.org/10.1596/978-1-4648-1559-1

APPENDICES

Appendix A: Project Links and How to Run the System

- Source code (MIT licence): https://github.com/tejirijesse/axk-open-source-capstone
- Hosted system (Stellar testnet): https://capstone.axk.org (click a demo account row on the sign-in page, then Sign in)
- Public verification: https://capstone.axk.org/trace
- Offline simulation (no blockchain, for trying the screens): https://axk-open-source-capstone.vercel.app
- Demonstration video: https://youtu.be/MrSXiJG_MAU

To run the full stack locally, install the tools listed in Table 10, then run `make toolchain`, `make install-app` and `make run-frappe`, and open http://localhost:8000/trade. To run all automated tests, run `make test`. The repository README lists every command, the required versions and the folder structure.

Appendix B: Evaluation Questionnaire Items

All scale items used a seven-point scale from 1 (strongly disagree) to 7 (strongly agree).

Table 19: Questionnaire scale items

Code	Item
PU1	Using this system would help me complete a trade faster.
PU2	Using this system would make it easier to prove the quality and origin of a lot.
PU3	Using this system would reduce the risk of not being paid, or not receiving what I paid for.
PU4	Overall, I would find this system useful in my work.
EOU1	Learning to use the system was easy for me.
EOU2	I could complete my task without help.
EOU3	The steps I had to follow were clear.
EOU4	Overall, I find the system easy to use.
T1	I trust that the inspection record cannot be changed after it is signed.
T2	I trust that payment will only be released to the right cooperative.
T3	I trust that I could check a lot's record myself without relying on the other party.
T4	I would trust a trading partner more if they used this system.
BI1	If this system were available, I would use it for my next cross-border trade.
BI2	I would recommend this system to other cooperatives or traders.
BI3	I would be willing to pay a small fee per trade to use it.

Profile questions covered the role used, a description of the participant's work, years in produce trade, whether they had traded across a border, mobile money use and main device. Baseline questions asked roughly how many days passed between agreeing the last deal and receiving the money or goods, what caused any delay and how payment was made. Open questions asked what was most useful, what would stop the participant or their cooperative from using the system, and what one thing they would change.

Appendix C: User Testing Report

Twenty participants completed the post-use questionnaire on 8 November 2026. Names and identifying details are removed; participants appear only as AXK1 to AXK20. Table 16 and Table 17 in Section 4.5.7 give the profile and construct scores; the item-level results and the reported causes of delay follow.

Table 20: Item-level questionnaire results (n = 20, scale 1 to 7)

Item	Mean	Lowest	Highest
PU1	6.35	5	7
PU2	6.40	5	7
PU3	6.05	3	7
PU4	6.40	5	7
EOU1	6.40	5	7
EOU2	6.20	4	7
EOU3	6.35	5	7
EOU4	6.45	5	7
T1	6.45	5	7
T2	6.50	5	7
T3	6.60	5	7
T4	6.65	5	7
BI1	6.80	5	7
BI2	6.85	6	7
BI3	6.85	6	7

Table 21: Causes of delay in participants' last trade (more than one could be chosen)

Cause	Participants	Percent
Quality dispute on arrival	15	75
Could not confirm the lot before paying	14	70
Currency or exchange	12	60
Waiting for payment to clear	11	55
Paperwork or certificates	10	50
Transport or border queue	9	45

Raw responses are held in the researcher's Google account, and signed consent forms are kept by the researcher in a separate Google Drive folder. Neither is included in this report.

Appendix D: Consent Form and Research Instruments

The following instruments were prepared for the study and submitted with the ethics application: the participant information sheet and consent form, the recruitment invitation, the cooperative permission letter, the semi-structured interview guide and the observation checklist. All 20 participants confirmed their consent in the questionnaire before answering, and the signed consent forms are kept by the researcher. Copies of the instruments are available from the researcher on request.

Appendix E: Smart Contract Security Guarantees

Table 22: Security guarantees enforced by the contracts and the tests that pin them

No.	Guarantee	Enforced by
1	No money moves for a lot that has not passed inspection	Escrow asks the registry for the approved cooperative before any transfer
2	The payee is the cooperative that registered the lot	Escrow creation has no payee argument
3	A release cannot redirect money	Release takes only a trade identifier
4	A lot identifier cannot be taken by another party	Lot id = SHA-256 of cooperative and lot reference
5	A cooperative cannot approve its own produce	Inspection refused when the inspector owns the lot
6	A failed lot cannot be re-inspected into a pass	Inspection is write-once
7	One inspected lot cannot be sold twice	One live escrow per lot
8	A buyer's refund does not depend on AXK	Refund needs no signature after the deadline
9	A trade identifier cannot be taken by a stranger	Creation needs both buyer and verifier
10	Roles cannot be claimed after deployment	Roles set in the constructor during deployment
11	A split cannot be rewritten or hijacked	Write-once split, coordinator only
12	Every unit of payment is accounted for	Rounding remainder paid to the last share
13	A settled trade's record cannot be rewritten	Attestation is write-once, attester only
14	Records do not expire before or during a trade	Storage lifetime extended on write and read; deadline capped